Not All Features Are Created Equal: A Mechanistic Study of Vision-Language-Action Models

Bryce Grant* **Xijia Zhao*** **Peng Wang†**
Case Western Reserve University
{bag100, xxz1277, pxw206}@case.edu
https://cwru-aism.github.io/vla-interp-page/

Abstract

A fine-tuned Vision-Language-Action (VLA) policy can place the alphabet soup in the basket on command, just as it saw during fine-tuning. However, if the basket is shifted five centimeters to the left, it drops the soup off the table. We developed an interpretability pipeline to investigate what the policy is computing. Injecting task A's activations into the action expert while the policy looks at task B's scene makes $\pi_{0.5}$ execute task A's motor trajectory in 99.6% of episodes (n=1,968). The effect is not universal, and the override falls to 52.1% on SmolVLA, where the interleaved fusion attenuates it, and to 57.0% on GR00T N1.5. The injected program is bound to absolute workspace coordinates and ignores the visible scene, which provides a mechanistic account of the perturbation brittleness that concurrent benchmarks report. The binding holds on a physical UR10e: directed replay redirects the commanded task in 26/26 rollouts in one direction and 18/26 in the other, with no-injection and random-direction controls at the floor. Once the source object is moved 10–15 cm, the arm grasps and releases at the recorded coordinates in 50/50 trials, picking up the wrong block when one sits there and closing on empty air when none does. Applying the same interventions to five VLAs ($\pi_{0.5}$, OpenVLA-OFT, X-VLA, SmolVLA, GR00T N1.5) and ACT as a language-free control, on LIBERO, MetaWorld, SimplerEnv, and ALOHA over 420,000+ rollouts, yields three findings: every architecture encodes the instruction, and four of the five weight it below vision whenever the scene alone identifies the goal; under the standard replacement write-back the SAE pooling that preserves behavior is architecture-specific ($\pi_{0.5}$ per-token, SmolVLA indifferent); and goal selection and motor generation separate into VLM and expert pathways wherever the two are distinguishable. We release **Action Atlas** (https://action-atlas.com), an interactive platform covering all 520+ trained SAEs, 79 identified concepts, and the scored UR10e rollouts. Code: https://github.com/CWRU-AISM/action-atlas.

1 Introduction

A Vision-Language-Action (VLA) policy [Zitkovich et al., 2023, Kim et al., 2024, Black et al., 2024, Physical Intelligence, 2025, Physical Intelligence et al., 2025] fine-tuned on LIBERO completes more than 90% of trials (Tab. 2). Move the target object a few centimeters and success collapses [Zhou et al., 2025, Fei et al., 2025], with the instruction unchanged and the object in plain view. Do VLAs follow the instruction and identify *what* to manipulate, or do they replay a coordinate-bound motor program that hits *where* the training distribution put the object? When a VLA-controlled robot

*Equal contribution.
†Corresponding author: pxw206@case.edu

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

misbehaves, operators have no principled way to diagnose the failure beyond behavioral observation, unlike classical robotics whose kinematic and control models are inspectable by construction [Craig, 2018].

Sparse autoencoders (SAEs) decompose the dense, polysemantic activations of language models into sparse, monosemantic features [Cunningham et al., 2023, Bricken et al., 2023, Templeton et al., 2024]. At scale they have surfaced safety-relevant representations [Templeton et al., 2024], and support activation steering without retraining [Turner et al., 2023, Rimsky et al., 2024]. Six concurrent studies have opened VLAs with these tools (Tab. 1): Häon et al. [2025] and Buurmeijer et al. [2026] steer scalar kinematics such as speed and height, Swann et al. [2026] and Jin et al. [2026] train SAEs on one policy and suppress or amplify single features, and Miao et al. [2026] raises success along a learned subspace. Each edits one direction inside one architecture with the task held fixed, so none can answer the question posed by the perturbation failures: when the policy acts, is it executing an instruction about an object or a trajectory toward a location? A single-feature edit cannot separate the two, because object and training-time location co-vary in every learned feature.

We separate the two by replacing the computation wholesale (Fig. 1). We record a policy's residual-stream activations while it performs task A in scene A, then write them into the forward pass while the same policy looks at scene B and is told to perform B. A task concept would be reinterpreted against scene B and send the arm to B's object. A coordinate-bound program would retrace A's path through B's scene. We run this test across five VLAs ($\pi_{0.5}$ [Physical Intelligence, 2025], OpenVLA-OFT [Kim et al., 2025], X-VLA [Zheng et al., 2026], SmolVLA [Shukor et al., 2025], GR00T N1.5 [Bjorck et al., 2025]) and one language-free control (ACT [Zhao et al., 2023]), spanning 80M to 7B parameters, four action-generation paradigms (flow matching, diffusion, continuous regression, and a conditional variational autoencoder, CVAE), and four benchmarks (420,000+ rollout episodes). Under this cross-task injection, $\pi_{0.5}$ follows the source task's trajectory in 99.6% of n=1,968 episodes while looking at the destination scene (Fig. 2). Per-model rates, each labeled by metric and reported tie-inclusively, are in Tab. 10. We describe this as a *coordinate-bound motor program*, or an activation-level computation that reproduces the source task's trajectory shape anchored to workspace coordinates when replayed in a new scene. This binding offers a mechanistic account of the brittleness pattern reported by Zhou et al. [2025].

Three additional findings repeat across the architectures measured. Language sensitivity depends on task structure. When vision identifies the goal, prompts carry less behavioral weight, even though linear probes read them off the residual stream almost perfectly (§4.4).

Pathway specialization separates goal selection in the VLM pathways from motor generation in the expert pathways wherever the two are separable. The dissociation is quantified on SmolVLA and reproduced directionally on $\pi_{0.5}$ and GR00T, and SmolVLA's interleaved fusion attenuates cross-task override on both LIBERO and MetaWorld (§4.5).

The SAE pooling preference differs across architectures. Under the standard replacement write-back $\pi_{0.5}$ and OFT require per-token reconstruction and SmolVLA is indifferent, and X-VLA and the GR00T diffusion transformer (DiT) tolerate mean-pooling when the reconstruction is written back as a residual. A temporal-contrastive arm degrades every architecture by collapsing adjacent latent codes (§4.6).

Our contributions include:

- A cross-architecture mechanistic analysis of six architectures (80M–7B) on four benchmarks under one intervention protocol. Each finding is scoped per model.
- Causal evidence for coordinate-bound motor programs. Cross-task injection (§4.3) and pathway specialization (§4.5) jointly localize the substrate, the localization explains the brittleness pattern of Zhou et al. [2025], and the program replicates on a physical UR10e (§4.7).
- Empirical guidelines for SAEs on VLA models. A three-arm comparison (per-token, mean-pool, temporal-contrastive) across five architectures shows that pooling preference is architecture-tied and that the temporal arm collapses adjacent latent codes ($\cos_{\text{adj}}$:$0.86 \rightarrow 0.997$) on every architecture.
- Action Atlas, an open-source interactive feature-exploration platform (https://action-atlas.com) covering all 520+ SAEs, 79 identified concepts, and 10,000+ rollout videos.

2 Related Work

RT-2 [Zitkovich et al., 2023] established VLAs by showing that VLMs can generate discrete-tokenized robot actions. OpenVLA-OFT later [Kim et al., 2025] replaced discrete tokenization with continuous L1 regression and achieved 97.1% LIBERO success. A parallel line of work dropped autoregressive decoding: $\pi_{0.5}$ [Physical Intelligence, 2025] introduced flow matching with a dedicated action expert, which avoids the bin-collapse failures of discrete tokens. The six models we study (Table 6) span this design space, from 80M (ACT) to 7B (OFT) parameters. Independent robustness evaluations [Fei et al., 2025, Zhou et al., 2025] have shown a pattern that our interpretability study explains mechanistically: small object-position perturbations drop OpenVLA and π_0 to 0.00 success across all four LIBERO suites and $\pi_{0.5}$ to 0.08–0.38 (vs. 0.92–0.98 on the unperturbed originals) [Zhou et al., 2025], while instruction-level perturbations drop all three to 0.00 on most suites despite passing semantic paraphrases. Together these failures are consistent with memorization-based behavior in which the instruction is encoded but not consulted.

SAE methods built for language models do not transfer to VLAs unchanged. Per-token preservation matters because action tokens carry phase structure absent in text (§4.6), and discrete-token action heads break SAE insertion (App. C.1).

Concurrent VLA-interpretability work is single-architecture and single-intervention. Häon et al. [2025] steer π_0/OpenVLA feedforward directions for kinematic behaviors such as speed and height on a real UR5. Swann et al. [2026] train SAEs on $\pi_{0.5}$ and OpenVLA and steer individual features, including object selection under an empty prompt, on DROID hardware. Buurmeijer et al. [2026] steer kinematics and object choice in $\pi_{0.5}$ with a linear observer-controller on DROID hardware. Miao et al. [2026] gate the action expert toward learned success subspaces, raising task success without changing which task is attempted. Jin et al. [2026] train event-grounded SAEs and report that feedforward-side semantic steering fails to control object choice. Khan et al. [2025] and Molinari et al. [2025] apply SAEs and world-model probes to single policies. Each study edits one direction in one architecture and holds the task fixed. Cross-task injection, the SAE pooling comparison, the pathway separation, and hardware replication are the axes this study adds (Tab. 1). Extended related work in App. A.

3 Method

We developed a closed-loop interpretability pipeline that makes rollout behavior the ground truth for every mechanistic claim (Fig. 1). Activation injection and counterfactual prompting intervene on the policy and read causality from the change in behavior: the former replays recorded computation inside the forward pass, and the latter changes the language input while the scene is held fixed. Then, to read out what the network encodes, SAEs decompose the recorded activations into features that localize the causal signal to specific layers and subspaces, and linear probes quantify what the residual stream carries at each layer [Li et al., 2026a]. Validation closes the loop. A feature surfaced by the SAEs counts as mechanistic only after it reproduces its effect through the same injection-and-rollout machinery.

Table 1: Our study and concurrent VLA-interpretability work. Arch.: architectures intervened on. FFN: feed-forward network. X-task: cross-task activation injection. Pool/Path: SAE pooling comparison and VLM-expert pathway split. HW: real robot.

Work	Arch.	Intervention	X-task	Pool/Path	HW
Häon et al. [2025]	2	FFN, kinematic	no	no/no	UR5
Swann et al. [2026]	2	SAE feature	no	no/no	DROID
Buurmeijer et al. [2026]	1	linear, kinematic + object	no	no/no	DROID
Miao et al. [2026]	3	subspace, success	no	no/no	DROID
Jin et al. [2026]	2	SAE, suppression	no	no/no	ALOHA
Khan et al. [2025]	1	SAE direction	no	no/no	sim
This work	**6**	**injection + SAE**	**yes**	**yes/yes**	**sim + UR10e**

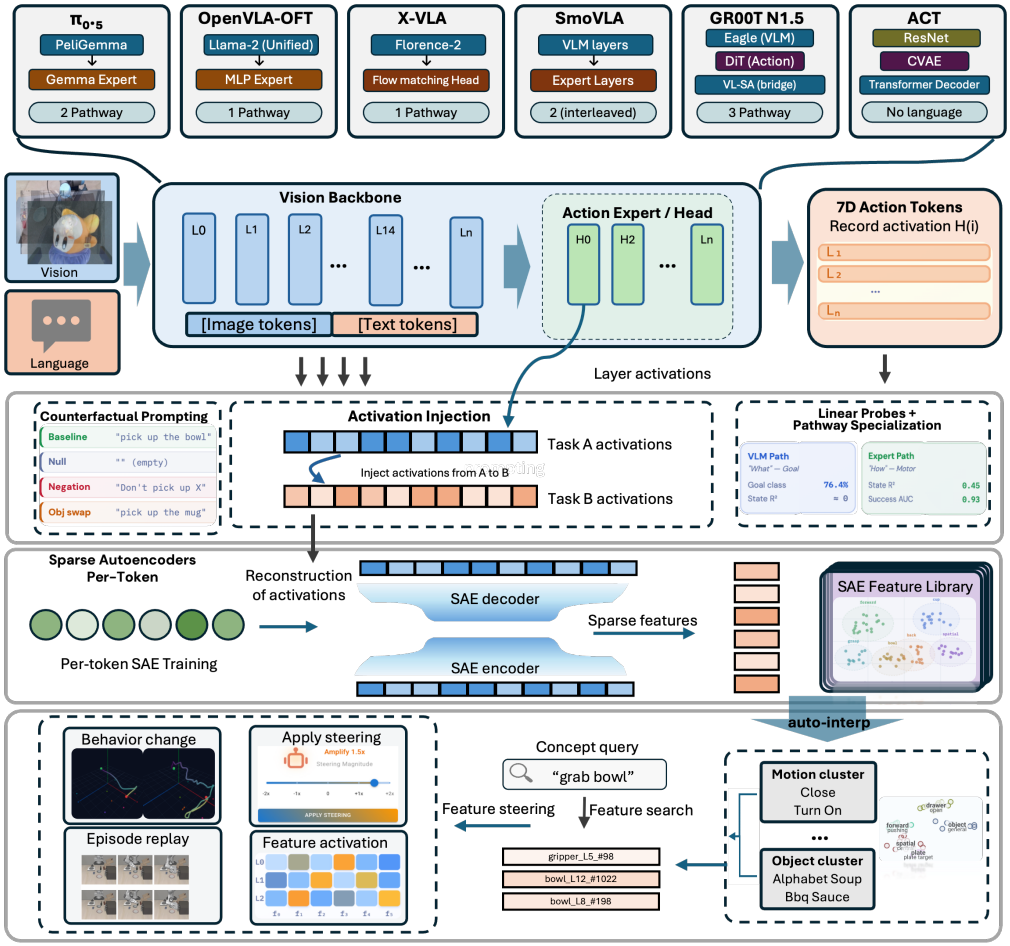


Figure 1: **Methodology overview.** Top: record a VLA's activations during a successful rollout and replay them under cross-task injection and counterfactual prompts. Middle: per-token SAEs decompose the same activations into features. Bottom: features are clustered, searched, and validated by ablation and steering in Action Atlas.

3.1 VLA Architectures Under Study

We study five VLAs and one language-free control: $\pi_{0.5}$ [Physical Intelligence, 2025] (3B, flow-matching), OpenVLA-OFT [Kim et al., 2025] (7B, continuous L1), X-VLA [Zheng et al., 2026] (1B, soft-prompted flow), SmolVLA [Shukor et al., 2025] (450M, interleaved VLM-expert), GR00T N1.5 [Bjorck et al., 2025] (3B, DiT-Eagle-VL-SA), and ACT [Zhao et al., 2023] (80M, CVAE, vision-only). Together they span 80M to 7B parameters and four action-generation paradigms: flow matching [Lipman et al., 2023], continuous regression, diffusion, and CVAE decoding (Tab. 6).

3.2 Activation Injection

Activation injection extends activation patching [Meng et al., 2022] to full rollout episodes: we replace activations from one episode with those from another during inference. Given source episode A (correct prompt, successful rollout) and target episode B (alternative condition), we record layer activations $\{\mathbf{H}^{A,(\ell)}\}$ during A, then at every timestep t of B override the residual stream at the chosen layers,

$$\mathbf{H}_t^{B,(\ell)} \leftarrow \mathbf{H}_t^{A,(\ell)}, \qquad \ell \in \mathcal{L}_{\text{inject}}, \tag{1}$$

and let the rest of the forward pass proceed normally. The injection set $\mathcal{L}_{\text{inject}}$ ranges from a single layer to all layers. When prompts A and B produce residual streams of different sequence lengths, we overwrite only the 50 action-token positions, which are fixed across prompt configurations. Image

and prompt tokens pass through unchanged. When the target runs longer than the recorded source ($T_B > T_A$), injection ceases at $t{=}T_A$ and the target policy continues unmodified (<5% of cross-task episodes; restricting to $T_B \leq T_A$ shifts override $\leq$1pp on $\pi_{0.5}$ `libero_goal`).

We test three conditions. In **null injection**, the source episode uses the correct prompt while the target uses an empty string. In **same-scene injection**, both episodes share the same visual scene but target different objects. In **cross-task injection**, source and target occupy different visual scenes. For multi-pathway models ($\pi_{0.5}$, SmolVLA, GR00T), we also inject into individual pathways to isolate their contributions.

3.3 Counterfactual Prompting

We vary text prompts to measure language sensitivity. Each episode is evaluated under one of six conditions: the baseline correct prompt, a null prompt (empty string), a negation prompt ("Don't pick up X"), a motor command ("Move slowly"), an object swap (replacing the target object name), and a temporal switch (changing the prompt mid-episode). For SmolVLA on MetaWorld, we also test counterfactual prompts across four difficulty levels (easy, medium, hard, very hard).

3.4 Sparse Autoencoders for VLAs

We train SAEs on action-relevant activations with TopK sparsity [Gao et al., 2024] ($k = 64$ active features) and expansion factor $m = 4d$ or $m = 8d$. Whether per-token preservation matters depends on architecture (§4.6). On $\pi_{0.5}$, mean-pooling across the 50 action tokens collapses approach, manipulation, and terminal phases into one vector and drops rollout success from 89.9% to 3.3% (grand mean over the four LIBERO suites; the per-layer `libero_object` sweep shows the same collapse, 93.8%→8.9%) despite normalized MSE ≤ 0.05 ($R^2 > 0.95$). We follow Gao et al. [2024] and report normalized MSE. Per-token processing is the conservative default, with per-architecture pooling effects in §4.6 and Appendix H.2. Concept-specific features are identified via frequency-weighted contrastive selection: $\text{score}_f = d_f \times \text{freq}_f$, where d_f is Cohen's d [Cohen, 1988] measuring activation difference between concept-present and concept-absent tasks, and freq_f is the fraction of samples where feature f appears in the active top-k. Across the five VLAs we train 520+ distinct SAEs (per-model counts in Tab. 7), which together identify 79 unique manipulation concepts (union of unique concept names across models; per-model counts overlap heavily) across motion, object, and spatial categories.

3.5 Linear Probes for Action Prediction

Linear probes [Alain and Bengio, 2017] test whether action information is linearly decodable from intermediate representations. We train ridge regression probes per action dimension and verify causality by projecting out the probe direction.

3.6 Metrics

We evaluate three primary metrics. **Action Cosine Similarity** measures behavioral alignment between episodes, $\cos(\mathbf{a}^X, \mathbf{a}^Y) = \langle \mathbf{a}^X, \mathbf{a}^Y \rangle / (\|\mathbf{a}^X\| \|\mathbf{a}^Y\|)$, computed over the full per-token action vectors. **Task Success** is a binary indicator determined by the environment's built-in success criteria. We report the change in success rate (SR) as $\Delta\text{SR} = \text{SR}_{\text{intervention}} - \text{SR}_{\text{baseline}}$. **Override Rate** quantifies how often the injected behavior dominates the prompted behavior in cross-task injection. Let $\Delta_t = \cos(\mathbf{a}_t^{B|A\text{-inj}}, \mathbf{a}_t^A) - \cos(\mathbf{a}_t^{B|A\text{-inj}}, \mathbf{a}_t^B)$. We define the override rate conditional on exclusion of a cosine-tie band $|\Delta_t| \leq \tau$,

$$\text{OR} \;=\; \Pr[\,\Delta_t > \tau \mid |\Delta_t| > \tau\,], \tag{2}$$

with $\tau = 0.05$. The share of steps inside the tie band varies by architecture, from 0.3% on $\pi_{0.5}$ to 60.0% on X-VLA, so we also report *tie-inclusive* override, $\Pr[\Delta_t > 0]$ over all injection steps, for every model (Tab. 10, Fig. 2).

4 Experiments

The experiments trace one causal chain: visual context selects a coordinate-bound motor program (§4.2, §4.3). Language then matters only when vision is ambiguous (§4.4), pathway specialization

implements that interface, the VLM side choosing the program and the expert side binding it to coordinates (§4.5), SAEs read out that substrate under a pooling choice that depends on architecture (§4.6), and the binding claim is tested on a physical arm (§4.7).

4.1 Experimental Setup

We collect 420,000+ rollout episodes across 12 experiment types and up to 50 tasks per environment (scale in Table 7), on 8×A100-80GB, RTX 5090, and 2×RTX 4090, over four benchmarks: **LIBERO** [Liu et al., 2023] (4 suites, 40 tasks), **MetaWorld** [Yu et al., 2020] (50 tasks, 4 difficulty levels), **SimplerEnv** [Li et al., 2024b] (10 tasks, 2 embodiments), and **ALOHA** [Zhao et al., 2023] (2 bimanual tasks).

4.2 Visual Context Dominates Action Generation

Visual context dominates action generation across all six architectures (Tab. 2). Injecting baseline visual-pathway activations under a null prompt restores cosine similarity to ≥ 0.99 and task success to 14–80%. Zeroing any single layer sends X-VLA and GR00T-DiT to 0%, while SmolVLA tolerates single expert-layer zeroing (19.2–76.5% across suites). Recovery is largest for dual-pathway designs ($\pi_{0.5}$ 70–80% per suite at n=50/suite, against matched null controls of 22%, 6.0% and 62%, Tab. 2) and smallest for single-pathway OFT (14–15%, n=120 per layer). We attribute the gap to visual-language entanglement, which carries more of the prompt-conditioning load in single-stack architectures (§4.4). The recovery is graded and scales with how much the prompt mattered. It restores 88% of the lost success on `libero_goal` and 69% on `libero_object`, while the `libero_spatial` contrast is not significant (p=0.13), the suite where an empty prompt costs 26%. Per-model layer-zeroing profiles in App. H.

Table 2: Visual-pathway influence per suite. Baselines come from a unified 25-ep×10-task batch (n=250/suite), and the OFT null prompt is n=50/suite. *One matched $\pi_{0.5}$ batch (n=50/suite) with its own baselines of 90.0%, 92.0% and 88.0% (App. H). **From the n=500 sweep of Tab. 4. Same-scene: baseline activations injected into a same-task episode. Rec.: OFT null prompt plus injected baseline activations. Zero-layer rows average the layer sweep. †The public GR00T spatial checkpoint underperforms the reference (App. K).

Model	Condition	Goal	Object	Spatial	Long / 10
$\pi_{0.5}$	Baseline	93.2%	87.6%	92.8%	86.0%
	Null prompt, no injection*	6.0%	22.0%	62.0%	56.6%**
	Null + PaliGemma-ALL inject*	80.0%	70.0%	78.0%	—
	Same-scene inject ALL	91.0%	87.5%	88.2%	75.2%
OFT	Baseline	98.8%	99.2%	92.4%	91.2%
	Null prompt, no injection	12.0%	96.0%	58.0%	—
	Null + inject zero-baseline (rec.)	14.1%	14.6%	14.4%	13.5%
X-VLA	Baseline	98.8%	98.8%	95.6%	87.6%
	Null prompt, no injection	10.0%	60.0%	48.0%	28.0%
	Zero any single layer	0.0%	0.0%	0.0%	0.0%
SmolVLA	Baseline	74.8%	87.6%	65.6%	41.2%
	Zero expert layer (mean)	71.0%	76.5%	61.8%	19.2%
GR00T N1.5	Baseline	93.6%	95.6%	69.2%†	77.2%
	Null source (inject ALL DiT)	3.3%	3.3%	—†	3.3%
	Zero any DiT layer (L0–L15)	0.0%	0.0%	—†	0.0%
ACT (ALOHA)	Baseline (n=100/task)	100.0% TransferCube, 78.0% Insertion			
	Mask workspace grid (2,2), n=20	10.0% TransferCube, 20.0% Insertion			

Within a shared scene, same-scene injection of prompt-B activations into a prompt-A rollout steers behavior toward the injected target on every multi-pathway model tested, and $\pi_{0.5}$ joins SmolVLA

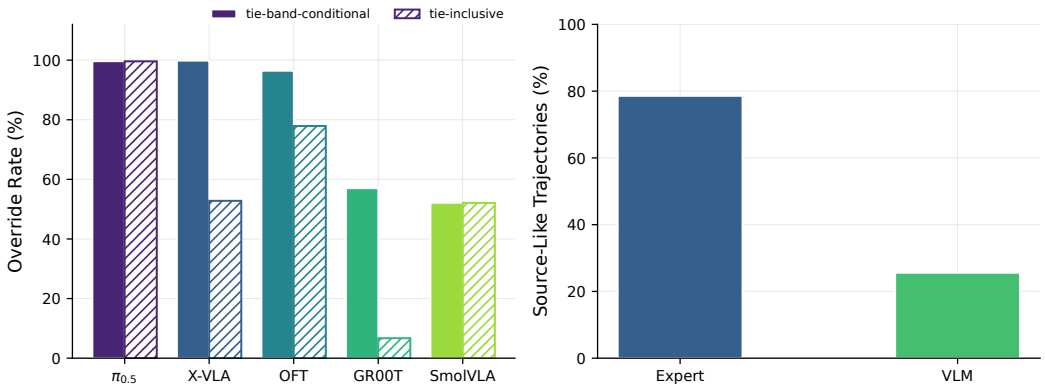


Figure 2: **Cross-task override rates.** Left: tie-band-conditional (solid) and tie-inclusive (hatched) override per model, with metrics and CIs in Tab. 10. Right: SmolVLA override by injected pathway on LIBERO (n=720).

in tolerating late-layer zeroing. Vision selects the action, and §4.3 asks what the selected activation encodes.

4.3 Motor Programs Bind to Absolute Spatial Coordinates

Does the injected activation encode a task concept ("pick up the cup") or a coordinate-bound motor program? Cross-task injection separates the two. The production condition replaces the expert-pathway residual stream at layers 16 and 17 at the 50 action-token positions on every step. Across all five language-capable VLAs, injection collapses destination-task success, and trajectory analysis shows the injected activations steer behavior toward source-task positions (Tab. 3; ACT is the language-free control).

Table 3: Cross-task injection across six models. Dst. SR: destination-task success under injection. Override rate: tie-band-conditional at τ=0.05 (Eq. 2), with tie-inclusive rates in Tab. 10. ACT's Dst. SR is its uninjected Insertion rate on the same 10 seeds.

Model	Pairs (n)	Dst. SR	Override rate	Notes
$\pi_{0.5}$	1,968	2.6%	99.6%	goal, spatial, object
OFT	1,079	5.0%	96.4%	90.8–100.0% per suite
X-VLA	3,150	0.0%	99.8%	LIBERO; WidowX 54.2%
SmolVLA	720	0.0%	52.1%	expert 78.6%, VLM 25.6%
GR00T	270	0.0%	57.0%	goal 85.6%, object 52.2%, long 33.3%
ACT (ctrl.)	10	70.0%	0.0%	no-op (cos =1.0)

Displacement analysis (Figure 2) shows that injection lowers task success and still steers behavior. $\pi_{0.5}$, X-VLA, and OFT are strongly source-dominant ($\geq$ 96% tie-band-conditional, 52.8–99.6% counting ties). GR00T is moderate (57.0% by action cosine, 6.7% by end-effector velocity), and SmolVLA's expert pathway overrides at 3.07$\times$ the rate of its VLM pathway. Objects do move under injection, but the moved object belongs to the destination scene, and DTW on end-effector trajectories gives 86–91% source dominance on `libero_goal` and `libero_spatial` (App. H.3).

A norm-matched random direction at the same layers gives chance-level override (App. C.3), so the injected direction carries the effect. Aligning source and destination by motion phase keeps the override at 94.4% (App. H.3), and it holds after a **tenfold cut** in fine-tuning data (App. N) and at half-strength injection (App. L). The robot reaches toward where the source objects would have been, and its motor program is bound to absolute workspace coordinates, consistent with Fei et al. [2025], Zhou et al. [2025]. This binding also replicates on hardware (§4.7).

4.4 Language Becomes Necessary Only When Vision is Ambiguous

If visual context selects the action, language should matter only when the scene fails to disambiguate the task. Counterfactual prompting across five language-capable VLAs (per-model n in App. H) confirms this in graded form: task structure sets the ordering of prompt tolerance, that ordering holds across architectures, and its absolute level varies by architecture. On the four LIBERO suites and four MetaWorld difficulty bands, every architecture we measure on both suites retains more success under null and wrong-prompt conditions on visually unambiguous tasks (`libero_object`, MetaWorld easy) than on visually ambiguous ones (`libero_goal`, MetaWorld hard), by a factor of two or more. $\pi_{0.5}$ null-prompt success is 31.6% on `libero_object` and 5.8% on `libero_goal`, against baselines of 91.0% and 92.6% (n=500/cell). $\pi_{0.5}$ ignores instruction semantics. Told `do not {task}`, it performs the task in 96.7% of episodes (n=30, within-run baseline 86.7%).

The level also varies by architecture and spans the full range on one suite. On `libero_object`, OFT retains 96.0% under a null prompt (n=50/cell), $\pi_{0.5}$ drops to 31.6%, and SmolVLA reaches 0.0% ($\pi_{0.5}$ and SmolVLA n=500/cell, against baselines of 100.0%, 91.0% and 89.2%). The LIBERO SmolVLA checkpoint fails all-or-nothing: 0/500 under a null prompt, and 0/12 under an empty, a one-token, and a nuisance prompt alike against 11/12 on the true instruction. However, the MetaWorld SmolVLA checkpoint keeps 82.0% under the same null condition on the easy band (Tab. 4).

Table 4: Counterfactual prompting across models (n per model in App. H). The n=500 $\pi_{0.5}$ and SmolVLA cells are 50 episodes per task from one fixed init state (LeRobot 0.4.4 [Cadene et al., 2024] pins it), paired on layout across conditions. ‡Matched within-experiment control, within 6 pp of the Tab. 2 baselines. §Not run on MetaWorld.

Model / Suite	Correct Prompt‡	Null	Wrong Obj.§
$\pi_{0.5}$ / object	91.0%	31.6%	—
$\pi_{0.5}$ / goal	92.6%	5.8%	—
$\pi_{0.5}$ / 10	86.4%	56.6%	—
$\pi_{0.5}$ / spatial	91.0%	55.0%	—
OFT / object	100.0%	96.0%	—
OFT / goal	98.0%	12.0%	—
OFT / spatial	92.0%	58.0%	—
X-VLA / object	100.0%	60.0%	60.0–90.0%
X-VLA / goal	94.0%	10.0%	4.0–10.0%
X-VLA / spatial	98.0%	48.0%	44.0–58.0%
SmolVLA / object	89.2%	0.0%	—
SmolVLA / MW easy	85.0%	82.0%	—
SmolVLA / MW hard	62.0%	41.0%	—
GR00T / object	93.0%	70.0%	50.0%
GR00T / goal	97.0%	0.0%	0.0%
GR00T / long	83.0%	67.0%	47.0%

Despite behavioral invariance, linear classifiers recover prompt category from late-stack activations at >99% accuracy on every language-capable architecture (Tab. 19). Every architecture encodes the prompt and consults it only when the scene leaves the goal open. Huang et al. [2026] report the same collapse on real hardware and attribute it to visual-encoder drift (App. A).

4.5 VLM Pathways Select the Goal and Expert Pathways Generate the Motor Program

The bind/select division of labor predicts an architectural division, with VLM pathways selecting the goal and motor-expert pathways generating the coordinate-bound motor program. The dissociation is quantified on SmolVLA and reproduced directionally on the other two multi-pathway models. Injecting expert pathways causes active misdirection (reaching to wrong locations), and injecting VLM pathways causes passive stalling (timeout) (Fig. 2, right). On SmolVLA, oracle state-prediction probes recover ground-truth state from expert activations at $4.5\times$ the VLM rate (oracle ratio 0.58 vs. 0.13, Tab. 19). The prediction has two halves, the VLM side decoding the goal better and the expert side decoding the state better. SmolVLA shows the state half and $\pi_{0.5}$ the goal half (a modest 14 pp gap): PaliGemma goal classification rises to 76.4% while expert goal classification settles at 62.6% (App. F.1), and GR00T DiT layers are more ablation-sensitive (40–80% drop) than VL-SA layers (Fig. 11), consistent with the same expert-encodes-state pattern under a different fusion. The goal

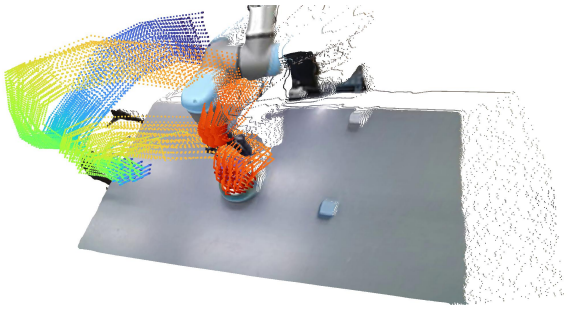


Figure 3: **Directed injection on the UR10e.** The plushie program is injected at expert L16/L17 under the prompt to stack the blocks. The arm reaches the plushie and carries it to the bowl, and the blocks stay untouched. Tracked points run from start (blue) to end (red).

side is causal. An edit to the VLM pathway at the instruction positions restores goal selection with no instruction on four of five architectures, and matched random or wrong-task edits do not. OFT, which has no separate VLM pathway, does not respond (App. G). Where a VLA has a separate expert, the expert holds the coordinates and the VLM holds the goal and an edit on the VLM side selects it.

4.6 Pooling Preference Is Architecture-Tied and Temporal Contrast Collapses Phase

Reading this substrate with SAEs requires a choice of how to pool across action tokens, and this choice depends on the architecture. We train TopK SAEs at each pathway layer (Eqs. 3–5; 520+ SAEs) and find two cross-architecture patterns.

Single-feature ablation has zero effect on 28–92% of (concept, task) pairs (Fig. 7), consistent with concepts spread redundantly across many features. Sensitivity concentrates in specific layers, and the action pathway behaves differently from the language stack (Fig. 5): the $\pi_{0.5}$ expert (18 layers) and GR00T DiT (16 layers) kill the policy (ΔSR ≥ 0.5) on $\geq$40% of ablation pairs across most of their depth, while OFT (32 L), X-VLA (24 L), and SmolVLA expert (32 L) keep the kill-switch rate below 10% almost everywhere. Width does not predict sensitivity (per-model profiles in Appendix H).

A five-architecture three-arm rollout comparison shows two pooling regimes (Tab. 12, rollout success through the per-token and mean-pool SAE). Mean-pooling collapses $\pi_{0.5}$ from 81% to 3% and OFT from 80% to 7%, so both require per-token reconstruction. X-VLA (51% to 80%) and GR00T-DiT (80% to 83%) do better under mean-pooling, and SmolVLA is indifferent (67.2% and 66.4%). The temporal-contrastive arm collapses adjacent codes ($\cos_{\text{adj}} : 0.86 \rightarrow 0.997$) at $R^2 \geq 0.97$ on $\pi_{0.5}$, X-VLA, and GR00T-DiT (SmolVLA 0.940, OFT data-limited). The collapse and the degradation co-occur on every architecture we ran. The guideline is to choose pooling per architecture and per signal (App. G), and to avoid the temporal-contrastive objective. Mean-pool numbers use the replacement write-back. A residual write-back removes the collapse on OFT, GR00T, and X-VLA (App. M).

4.7 The Program Replicates on a Physical UR10e

To test the binding claim of §4.3 off the simulator, we fine-tuned $\pi_{0.5}$ on a Universal Robots UR10e over seven tabletop tasks (76.2% baseline, 160/210, 30 rollouts per task) and ran the same wholesale expert replay at L16/L17 between plushie-in-bowl and gray-on-blue stacking, in both directions (App. O). Directed injection (Tab. 5) redirects the commanded task in 26/26 rollouts in one direction and 18/26 in the other, against matched-scene no-injection floors of 0/24 and 0/25 and a norm-matched random direction at 0/25 and 1/23. A displaced-object test then separates object tracking from coordinate anchoring: with the source object moved 10–15 cm off its recorded position, the injected program drives the arm to the recorded coordinates in 50/50 trials across both directions, while the uninjected policy reaches the displaced object (demonstrated in Fig. 3, used CoTracker3 [Karaev et al., 2024] and DA3 [Lin et al., 2025]). The physical replay also reuses the source's flow-matching sampling noise, which the simulation runs leave unmatched (App. O).

Table 5: Physical cross-task injection on the UR10e, source-classified rate per condition [95% CI].

Condition	Blocks program under plushie prompt	Plushie program under blocks prompt
Directed injection	**26/26** = 100.0% [87.1, 100.0]	**18/26** = 69.2% [50.0, 83.5]
No injection (floor)	0/24 = 0.0% [0.0, 13.8]	0/25 = 0.0% [0.0, 13.3]
Norm-matched random	0/25 = 0.0% [0.0, 13.3]	1/23 = 4.3% [0.8, 21.0]

4.8 Limitations

Simulation carries all cross-architecture results, and hardware covers a single model and task pair (§4.7). Extended limitations and three falsifiable predictions are in App. B.

5 Conclusion

Across six architectures, fine-tuned VLAs encode coordinate-bound motor programs that the visual pathway selects and, where a separate expert exists, the expert pathway executes. Injected at the action expert, a source task's activations carry its trajectory into a different scene in 99.6% of $\pi_{0.5}$ episodes, and language sensitivity is architecture- and task-dependent (§4.4). The binding accounts for benchmark brittleness [Zhou et al., 2025, Fei et al., 2025] and replicates on a physical UR10e (§4.7). The goal lives on the VLM side, where one edit restores it with no instruction on four of five architectures. For SAEs on VLAs, pooling is an architecture-level choice, and the temporal-contrastive objective degrades every architecture ran.

Acknowledgments and Disclosure of Funding

This material is based upon work supported by the National Science Foundation Graduate Research Fellowship under Grant No. 2437833 and by National Science Foundation Grant No. 2533634. The authors also thank NVIDIA for computing resources provided through the NVIDIA Academic Grant Program.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Pierre Sermanet, Nicolas Tomas, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, and Andy Zeng. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022. doi: 10.48550/arXiv.2204.01691.

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *International Conference on Learning Representations (Workshop)*, 2017. URL https://arxiv.org/abs/1610.01644.

ALOHA 2 Team, Jorge Aldaco, Travis Armstrong, Chelsea Finn, Pete Florence, Jonathan Tompson, and Tony Z. Zhao. ALOHA 2: An enhanced low-cost hardware for bimanual teleoperation. *arXiv preprint arXiv:2405.02292*, 2024. URL https://arxiv.org/abs/2405.02292.

Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. doi: 10.1162/coli_a_00422.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015. URL https://arxiv.org/abs/1506.03099.

Usha Bhalla, Alex Oesterling, Claudio Mayrink Verdun, Himabindu Lakkaraju, and Flavio P. Calmon. Temporal sparse autoencoders: Leveraging the sequential nature of language for interpretability. *arXiv preprint arXiv:2511.05541*, 2025. URL https://arxiv.org/abs/2511.05541. ICLR 2026 Oral.

Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. URL https://arxiv.org/abs/2503.14734.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. doi: 10.48550/arXiv.2410.24164. URL https://arxiv.org/abs/2410.24164.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemantic-features/index.html.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. URL https://arxiv.org/abs/2212.06817.

Hugo Buurmeijer, Carmen Amo Alonso, Aiden Swann, and Marco Pavone. Observing and controlling features in vision-language-action models, 2026.

Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. https://github.com/huggingface/lerobot, 2024.

Chilam Cheang, Sijin Chen, Zhongren Cui, Yingdong Hu, Liqun Huang, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Xiao Ma, Hao Niu, Wenxuan Ou, Wanli Peng, Zeyu Ren, Haixin Shi, Jiawen Tian, Hongtao Wu, Xin Xiao, Yuyang Xiao, Jiafeng Xu, and Yichu Yang. GR-3 technical report. *arXiv preprint arXiv:2507.15493*, 2025. URL https://arxiv.org/abs/2507.15493.

Jiayi Chen, Wenxuan Song, Jiaxin Fang, Ruiqing Yin, Jingbo Wang, Shuai Chen, Jieyuan Pei, Yikai Qin, Feifan Chen, Haodong Yan, Zhide Zhong, Wen Chen, Yan Wang, Yuxiang Gao, and Haoang Li. Dfm-vla: Iterative action refinement for robot manipulation via discrete flow matching, 2026. URL https://arxiv.org/abs/2603.26320.

Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL https://arxiv.org/abs/2303.04137.

Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.

John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson, Hoboken, NJ, 4 edition, 2018. ISBN 9780133489798.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023. URL https://arxiv.org/abs/2309.08600.

Yinpei Dai, Jayjun Lee, Nima Fazeli, and Joyce Chai. RACER: Rich language-guided failure recovery policies for imitation learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025. URL https://arxiv.org/abs/2409.14674.

Shaoqi Dong, Chaoyou Fu, Haihan Gao, Yi-Fan Zhang, Chi Yan, Chu Wu, Xiaoyu Liu, Yunhang Shen, Jing Huo, Deqiang Jiang, Haoyu Cao, Yang Gao, Xing Sun, Ran He, and Caifeng Shan. VITA-VLA: Efficiently teaching vision-language models to act via action expert distillation. *arXiv preprint arXiv:2510.09607*, 2025. URL https://arxiv.org/abs/2510.09607.

Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. PaLM-E: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023. doi: 10.48550/arXiv.2303.03378.

Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371*, 2024. URL https://arxiv.org/abs/2410.00371.

Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025. URL https://arxiv.org/abs/2510.13626.

Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024. doi: 10.48550/arXiv.2406.04093. URL https://arxiv.org/abs/2406.04093.

Davide Ghilardi, Federico Belotti, Marco Molinari, Tao Ma, and Matteo Palmonari. Group-SAE: Efficient training of sparse autoencoders for large language models via layer groups. *arXiv preprint arXiv:2410.21508*, 2024. URL https://arxiv.org/abs/2410.21508.

Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. CAST: Counterfactual labels improve instruction following in vision-language-action models. *arXiv preprint arXiv:2508.13446*, 2025. URL https://arxiv.org/abs/2508.13446.

Michal Golovanevsky, William Rudman, Michael Lepori, Amir Bar, Ritambhara Singh, and Carsten Eickhoff. Pixels versus priors: Controlling knowledge priors in vision-language models through visual counterfacts. *arXiv preprint arXiv:2505.17127*, 2025. URL https://arxiv.org/abs/2505.17127.

Bear Häon, Kaylene Stocking, Ian Chuang, and Claire Tomlin. Mechanistic interpretability for steering vision-language-action models. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 2743–2762. PMLR, 2025. URL https://arxiv.org/abs/2509.00328.

Yutong Hu, Jan-Nico Zaech, Nikolay Nikolov, Yuanqi Yao, Sombit Dey, Giuliano Albanese, Renaud Detry, Luc Van Gool, and Danda Paudel. AR-VLA: True autoregressive action expert for vision–language–action models. *arXiv preprint arXiv:2603.10126*, 2026. URL https://arxiv.org/abs/2603.10126.

Suning Huang, Jiaqi Shao, Ke Wang, Qianzhong Chen, Jiankai Sun, Yanjiang Guo, Mac Schwager, and Jeannette Bohg. Breaking lock-in: Preserving steerability under low-data vla post-training. *arXiv preprint arXiv:2604.23121*, 2026. URL https://arxiv.org/abs/2604.23121. Concurrent work.

Yuhua Jiang, Shuang Cheng, Yan Ding, Feifei Gao, and Biqing Qi. AsyncVLA: Asynchronous flow matching for vision–language–action models. *arXiv preprint arXiv:2511.14148*, 2025. URL https://arxiv.org/abs/2511.14148.

Xinchen Jin, Aditya Chatterjee, Pranav Kumar, and Rohan Paleja. Event-grounded sparse autoencoders for vision-language-action policies, 2026.

Sonia Joseph, Praneet Suresh, Ethan Goldfarb, Lorenz Hufe, Yossi Gandelsman, Robert Graham, Danilo Bzdok, Wojciech Samek, and Blake Aaron Richards. Steering clip's vision transformer with sparse autoencoders, 2025. URL https://arxiv.org/abs/2504.08729.

Nikita Karaev, Iurii Makarov, Jianyuan Wang, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. Cotracker3: Simpler and better point tracking by pseudo-labelling real videos. *arXiv preprint arXiv:2410.11831*, 2024.

Chiraag Kaushik, Davis Barch, and Andrea Fanelli. Decomposing multimodal embedding spaces with group-sparse autoencoders. *arXiv preprint arXiv:2601.20028*, 2026. URL https://arxiv.org/abs/2601.20028.

M. A. Khan, N. Boskov, F. M. Anwar, and M. A. Khan. Controlling vision-language-action policies through sparse latent directions. In *NeurIPS 2025 Workshop on Mechanistic Interpretability*, 2025. URL https://openreview.net/pdf?id=wtf3ww1EOL.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024. doi: 10.48550/arXiv.2406.09246. URL https://arxiv.org/abs/2406.09246.

Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Robotics: Science and Systems (RSS)*, 2025. URL https://arxiv.org/abs/2502.19645.

Yu Lei, Minghuan Liu, Abhiram Maddukuri, Zhenyu Jiang, and Yuke Zhu. A mechanistic analysis of sim-and-real co-training in generative robot policies. *arXiv preprint arXiv:2604.13645*, 2026. URL https://arxiv.org/abs/2604.13645.

Haoxuan Li, Zhen Wen, Qiqi Jiang, Chenxiao Li, Yuwei Wu, Yuchen Yang, Yiyao Wang, Xiuqi Huang, Minfeng Zhu, and Wei Chen. Conceptviz: A visual analytics approach for exploring concepts in large language models. *IEEE Transactions on Visualization and Computer Graphics*, 32(1):57–67, 2026a. doi: 10.1109/TVCG.2025.3634806.

Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. SimpleVLA-RL: Scaling VLA training via reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026b. URL https://arxiv.org/abs/2509.09674.

Jinming Li, Yichen Zhu, Zhibin Tang, Junjie Wen, Minjie Zhu, Xiaoyu Liu, Chengmeng Li, Ran Cheng, Yaxin Peng, Yan Peng, and Feifei Feng. CoA-VLA: Improving vision-language-action models via visual-textual chain-of-affordance. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. URL https://arxiv.org/abs/2412.20451.

Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, Xiaofan Wang, Bei Liu, Jianlong Fu, Jianmin Bao, Dong Chen, Yuanchun Shi, Jiaolong Yang, and Baining Guo. CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024a. URL https://arxiv.org/abs/2411.19650.

Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishika Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. In *Proceedings of The 8th Conference on Robot Learning (CoRL)*, 2024b. URL https://simpler-env.github.io/. SIMPLER: simulation-based evaluation for real-world policies.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023. URL https://arxiv.org/abs/2209.07753.

Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2023. URL https://arxiv.org/abs/2210.02747. Alias for lipman2023flow.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://arxiv.org/abs/2306.03310.

Jiaming Liu, Hao Chen, Pengju An, Zhuoyang Liu, Renrui Zhang, Chenyang Gu, Xiaoqi Li, Ziyu Guo, Sixiang Chen, Mengzhen Liu, Chengkai Hou, Mengdi Zhao, KC alex Zhou, Pheng-Ann Heng, and Shanghang Zhang. Hybridvla: Collaborative diffusion and autoregression in a unified vision-language-action model, 2025a. URL https://arxiv.org/abs/2503.10631.

Zhenyang Liu, Yongchong Gu, Sixiao Zheng, Yanwei Fu, Xiangyang Xue, and Yu-Gang Jiang. TriVLA: A triple-system-based unified vision-language-action model with episodic world modeling for general robot control. *arXiv preprint arXiv:2507.01424*, 2025b. URL https://arxiv.org/abs/2507.01424.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. URL https://arxiv.org/abs/2202.05262.

Miranda Muqing Miao, Subin Kim, Brandon Yang, and Lyle Ungar. Contrastive conceptor activation steering (COAST): Unlocking vision-language-action models through hidden states, 2026.

Marco Molinari, Leonardo Nevali, Saharsha Navani, and Omar G. Younis. Emergent world representations in OpenVLA. *arXiv preprint arXiv:2509.24559*, 2025. doi: 10.48550/arXiv.2509.24559. URL https://arxiv.org/abs/2509.24559.

Aaron Mueller. Missed causes and ambiguous effects: Counterfactuals pose challenges for interpreting neural networks. *arXiv preprint arXiv:2407.04690*, 2024. URL https://arxiv.org/abs/2407.04690.

Ali Nasiri-Sarvi, Hassan Rivaz, and Mahdi S. Hosseini. SPARC: Concept-aligned sparse autoencoders for cross-model and cross-modal interpretability. *arXiv preprint arXiv:2507.06265*, 2025. URL https://arxiv.org/abs/2507.06265.

Bruno A. Olshausen and David J. Field. Sparse coding with an overcomplete basis set: A strategy employed by V1? *Vision Research*, 37(23):3311–3325, 1997. doi: 10.1016/S0042-6989(97)00169-7.

Mateusz Pach, Shyamgopal Karthik, Quentin Bouniot, Serge Belongie, and Zeynep Akata. Sparse autoencoders learn monosemantic features in vision-language models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025. URL https://arxiv.org/abs/2504.02821.

Karl Pertsch, Kyle Walke, Oier Mees, Chelsea Finn, and Sergey Levine. FAST: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025. URL https://arxiv.org/abs/2501.09747.

Physical Intelligence. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. URL https://arxiv.org/abs/2504.16054.

Physical Intelligence. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026. URL https://arxiv.org/abs/2604.15483.

Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Chelsea Finn, Sergey Levine, et al. $\pi^*_{0.6}$: a VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025. URL https://arxiv.org/abs/2511.14759.

Joris Postmus and Steven Abreu. Steering large language models using conceptors: Improving addition-based activation engineering. *arXiv preprint arXiv:2410.16314*, 2024. URL https://arxiv.org/abs/2410.16314. Workshop on Foundation Model Interventions at NeurIPS 2024.

Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, and Xuelong Li. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems (RSS)*, 2025. URL https://arxiv.org/abs/2501.15830.

Moritz Reuss, Hongyi Zhou, Marcel Rühle, Ömer Erdinç Yağmurlu, Fabian Otto, and Rudolf Lioutikov. FLOWER: Democratizing generalist robot policies with efficient vision-language-flow models. In *Proceedings of The 9th Conference on Robot Learning (CoRL)*, volume 305 of *Proceedings of Machine Learning Research*, pages 3736–3761. PMLR, 2025. URL https://arxiv.org/abs/2509.04996.

Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15504–15522, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL https://aclanthology.org/2024.acl-long.828/.

Stéphane Ross, Geoffrey J Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635. PMLR, 2011. URL https://proceedings.mlr.press/v15/ross11a.html.

Som Sagar, Jiafei Duan, Sreevishakh Vasudevan, Yifan Zhou, Heni Ben Amor, Dieter Fox, and Ransalu Senanayake. RoboFail: Analyzing failures in robot learning policies. *arXiv preprint arXiv:2412.02818*, 2024. URL https://arxiv.org/abs/2412.02818.

Haoyuan Shi, Xiancong Ren, Yingji Zhang, Qinfan Zhang, Jiayu Hu, Haozhe Shan, Han Dong, Jinpeng Lu, Yinda Chen, Yi Zhang, Yong Dai, and Xiaozhu Ju. VLA-Trace: Diagnosing vision-language-action models through representation and behavior tracing, 2026.

Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. URL https://arxiv.org/abs/2506.01844.

Anushka Sivakumar, Andrew Zhang, Zaber Ibn Abdul Hakim, and Chris Thomas. SteerVLM: Robust model control through lightweight activation steering for vision language models. *arXiv preprint arXiv:2510.26769*, 2025. URL https://arxiv.org/abs/2510.26769.

Samuel Soo, Guang Chen, Wesley Teng, Chandrasekaran Balaganesh, Guoxian Tan, and Ming Yan. Interpretable steering of large language models with feature guided activation additions. *arXiv preprint arXiv:2501.09929*, 2025. URL https://arxiv.org/abs/2501.09929. Building Trust Workshop at ICLR 2025.

Samuel Stevens, Wei-Lun Chao, Tanya Berger-Wolf, and Yu Su. Interpretable and testable vision features via sparse autoencoders. *arXiv preprint arXiv:2502.06755*, 2025. URL https://arxiv.org/abs/2502.06755. Introduces saev package for training SAEs on ViTs.

Aiden Swann, Lachlain McGranahan, Hugo Buurmeijer, Monroe Kennedy III, and Mac Schwager. Sparse autoencoders reveal interpretable and steerable features in VLA models. In *Conference on Robot Learning (CoRL)*, 2026.

Shuhan Tan, Kairan Dou, Yue Zhao, and Philipp Krähenbühl. Interactive post-training for vision-language-action models. *arXiv preprint arXiv:2505.17016*, 2025. URL https://arxiv.org/abs/2505.17016.

Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C Daniel Freeman, Theodore R Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html.

Harrish Thasarathan, Julian Forsyth, Thomas Fel, Matthew Kowal, and Konstantinos G. Derpanis. Universal sparse autoencoders: Interpretable cross-model concept alignment. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, pages 59304–59325, 2025. URL https://arxiv.org/abs/2502.03714.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization. *arXiv preprint arXiv:2308.10248*, 2023. doi: 10.48550/arXiv.2308.10248. URL https://arxiv.org/abs/2308.10248.

Junjie Wen, Minjie Zhu, Jiaming Liu, Zhiyuan Liu, Yicun Yang, Linfeng Zhang, Shanghang Zhang, Yichen Zhu, and Yi Xu. dVLA: Diffusion vision-language-action model with multimodal chain-of-thought. *arXiv preprint arXiv:2509.25681*, 2025. URL https://arxiv.org/abs/2509.25681.

Sihao Wu, Gaojie Jin, Wei Huang, Jianhong Wang, and Xiaowei Huang. Activation steering meets preference optimization: Defense against jailbreaks in vision language models. *arXiv preprint arXiv:2509.00373*, 2025. doi: 10.48550/arXiv.2509.00373. URL https://arxiv.org/abs/2509.00373.

Theodor Wulff, Sergio Lanza, Tamara Bila, Angelo Cangelosi, Stefan Wermter, and Igor Farkas. Latent cluster analysis for vision-language-action models, 2026.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2020. URL https://arxiv.org/abs/1910.10897.

Yifu Yuan, Haiqin Cui, Yaoting Huang, Yibin Chen, Fei Ni, Zibin Dong, Pengyi Li, Yan Zheng, and Jianye Hao. Embodied-R1: Reinforced embodied reasoning for general robotic manipulation. *arXiv preprint arXiv:2508.13998*, 2025. URL https://arxiv.org/abs/2508.13998.

Vladimir Zaigrajew, Hubert Baniecki, and Przemyslaw Biecek. Interpreting clip with hierarchical sparse autoencoders, 2025. URL https://arxiv.org/abs/2502.20578.

Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. URL https://arxiv.org/abs/2403.03954.

Jianke Zhang, Xiaoyu Chen, Qiuyue Wang, Mingsheng Li, Yanjiang Guo, Yucheng Hu, Jiajun Zhang, Shuai Bai, Junyang Lin, and Jianyu Chen. VLM4VLA: Revisiting vision-language-models in vision-language-action models. *arXiv preprint arXiv:2601.03309*, 2026. URL https://arxiv.org/abs/2601.03309.

Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Ming-Yu Liu, Donglai Xiang, Gordon Wetzstein, and Tsung-Yi Lin. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. URL https://arxiv.org/abs/2503.22020.

Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. URL https://arxiv.org/abs/2304.13705.

Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang, Jingjing Liu, Tai Wang, and Xianyuan Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2510.10274.

Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025. URL https://arxiv.org/abs/2510.03827.

Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol

Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

App.	Contents	Supports
A	VLA architectures, interpretability, robustness benchmarks	§2
B	Limitations and falsifiable predictions	§4.3, §4.8
C	Controls: discrete-token OpenVLA, ACT, random-direction injection	§4.3
D	SAE training, concept scoring, protocols, architectures, benchmarks, compute	§3
E	Cross-model radar and the per-architecture pathway metrics	§4.5
F	Goal-encoding dimensions, FFN neurons, layer similarity on $\pi_{0.5}$	§4.5
G	Goal steering with no instruction, on five architectures	§4.5
H	Tie-inclusive override, steering sensitivity, SAE pooling arms, DTW	§4.2–§4.6
I	GR00T phase-conditioned feature ablation	§4.6
J	Qualitative rollouts and additional figures	all
K	Per-architecture failure modes, OFT per suite, cross-model probing	§4.2–§4.4
L	Dose-response of expert injection	§4.3
M	SAE sparsity sweep and residual write-back	§4.6
N	Override under a tenfold cut in fine-tuning data	§4.3
O	UR10e setup, directed injection, displaced-object test	§4.7
P	Broader impact	

A Extended Related Work

A.1 Vision-Language-Action Models

RT-2 [Zitkovich et al., 2023] showed that web-pretrained VLMs can be fine-tuned for manipulation via tokenized actions. OpenVLA [Kim et al., 2024] extended this with an open-source 7B model, and OpenVLA-OFT [Kim et al., 2025] replaced discrete tokenization with continuous L1 regression, achieving 97.1% LIBERO success. The π_0, $\pi_{0.5}$, $\pi^*_{0.6}$, and $\pi_{0.7}$ models [Black et al., 2024, Physical Intelligence, 2025, Physical Intelligence et al., 2025, Physical Intelligence, 2026] introduced flow matching with a dedicated action expert and progressively more diverse training context. Whether $\pi_{0.7}$'s reported emergent capabilities reflect compositional generalization or further-scaled memorized motor patterns is an unexplored question that these mechanistic tools could possibly explain. Current VLAs span several scales and paradigms: SmolVLA [Shukor et al., 2025] (450M), X-VLA [Zheng et al., 2026] (cross-embodiment), GR00T N1.5 [Bjorck et al., 2025] (humanoid), CogACT [Li et al., 2024a], GR-3 [Cheang et al., 2025], VITA-VLA [Dong et al., 2025], SimpleVLA-RL [Li et al., 2026b], RIPT-VLA [Tan et al., 2025], FLOWER [Reuss et al., 2025], SpatialVLA [Qu et al., 2025], and ACT [Zhao et al., 2023] (80M, no language). Modular approaches include SayCan [Ahn et al., 2022], PaLM-E [Driess et al., 2023], and Code as Policies [Liang et al., 2023]. Concurrent representation-analysis work in robot policies uses different tools: Lei et al. [2026] apply UMAP and Wasserstein-distance analysis to characterize sim-real co-training, identifying "structured representation alignment" and "importance reweighting" effects. Our SAE-based feature-level analysis is complementary.

Action Representation. Diffusion Policy [Chi et al., 2023] treats action generation as conditional denoising, 3D Diffusion Policy [Ze et al., 2024] incorporates 3D representations, and FAST [Pertsch et al., 2025] uses DCT-based compression tokenization. Action tokenization directly determines SAE applicability (Section C.1).

Language Following. CAST [Glossop et al., 2025] augments robot datasets with counterfactual language labels. VLM4VLA [Zhang et al., 2026] found that VLM benchmark performance does not predict VLA task success, so language understanding and visuomotor competence are decoupled.

A.2 Mechanistic Interpretability

SAEs [Olshausen and Field, 1997, Bricken et al., 2023, Cunningham et al., 2023, Gao et al., 2024, Templeton et al., 2024] decompose dense activations into sparse interpretable features, with extensions to vision [Stevens et al., 2025, Joseph et al., 2025, Zaigrajew et al., 2025] and vision-language models [Pach et al., 2025]. Activation steering [Turner et al., 2023, Rimsky et al., 2024] enables behavioral control, with advances including conceptor-based steering [Postmus and Abreu, 2024], SAE-guided additions [Soo et al., 2025], and VLM safety steering [Wu et al., 2025, Sivakumar et al., 2025]. Linear probing [Alain and Bengio, 2017, Belinkov, 2022] identifies linearly decodable information, and establishing causal relevance requires intervention [Mueller, 2024]. PvP [Golovanevsky et al., 2025] shows vision and priors compete in multimodal models.

A.3 Interpretability for Robot Learning

RT-1 [Brohan et al., 2022] included attention visualizations, Diffusion Policy [Chi et al., 2023] analyzed action distributions, and ALOHA [Zhao et al., 2023, ALOHA 2 Team et al., 2024] demonstrated bimanual manipulation, all without internal representation analysis. RoboFail [Sagar et al., 2024], AHA [Duan et al., 2024], and RACER [Dai et al., 2025] characterize failures behaviorally. Häon et al. [2025] introduced VLA steering via internal representations, Molinari et al. [2025] probed for emergent world models, and Khan et al. [2025] used SAEs to isolate interpretable steering directions in Magma. Interpretability-by-design approaches include CoA-VLA [Li et al., 2025], CoT-VLA [Zhao et al., 2025], TriVLA [Liu et al., 2025b], dVLA [Wen et al., 2025], and Embodied-R1 [Yuan et al., 2025].

Cross-Model and Multimodal SAE Alignment. Per-model SAE training, including ours, leaves features in incompatible per-architecture spaces, which complicates direct comparison of concepts across pathways or models. Two recent lines of work address this. The first scales SAEs across model dimensions: Universal Sparse Autoencoders [Thasarathan et al., 2025] and SPARC [Nasiri-Sarvi et al., 2025] learn a single shared dictionary across multiple networks, enforcing concept alignment via shared encoders or a global TopK constraint, while Group-SAE [Ghilardi et al., 2024] reduces training cost by tying nearby layers into a single dictionary using residual-stream similarity. The second targets multimodal alignment: Kaushik et al. [2026] address the "split dictionary" phenomenon (where features fire only for one modality) using cross-modal random masking and group-sparse regularization. The split-dictionary problem mirrors our VLM/expert-pathway split (§4.5), and a multimodal-aligned SAE can test whether the goal/motor dissociation is orthogonal at the dictionary level or an artifact of independent training. We leave this extension to future work.

Concurrent VLA Architectures. Several recent VLA designs explore alternative action-decoding paradigms whose binding behavior our framework can probe directly. On the autoregressive side, AR-VLA [Hu et al., 2026] treats action generation as a true causal sequence with a long-lived action-history memory, breaking from chunk-based reactive heads. On the flow-matching side, DFM-VLA [Chen et al., 2026] replaces fixed-token decoding with discrete flow-matching iterative refinement, and AsyncVLA [Jiang et al., 2025] introduces asynchronous flow matching with a confidence rater for selective regeneration. Bridging the two paradigms, HybridVLA [Liu et al., 2025a] unifies discrete autoregression and continuous diffusion within a single LLM backbone. Each design changes the pathway through which an action is produced from a visual scene, but the cross-task injection and pathway-zeroing protocols in this paper transfer mechanically. We document our concern with the hook placement choice in App. D.4. Whether the coordinate-bound binding documented here persists under iterative-refinement (DFM-VLA), confidence-aware regeneration (AsyncVLA), or context-aware autoregression (AR-VLA) is an extension we leave to future work. We expect the binding to weaken in proportion to how much an architecture re-grounds on the current observation between action chunks, with no commitment to a precomputed plan.

Single-Policy Diagnostics. VLA-Trace [Shi et al., 2026] traces $\pi_{0.5}$ and OpenVLA across checkpoints with kernel alignment and attention knockout and reports strong visual grounding alongside weak fine-grained instruction following, the behavioral pattern of §4.4. LAVLA [Wulff et al., 2026] clusters GR00T N1.5 action-decoder latents layer by layer and finds spatiotemporal and kinematic structure separating in the middle layers. Both analyze one or two policies and do not intervene on task identity. Huang et al. [2026] report the same position-induced collapse of a fine-tuned $\pi_{0.5}$ on hardware and attribute it to visual-encoder drift. Our probes find grounding intact (>99% decodable), which places the failure downstream in the action pathway.

B Limitations

Predictions. If the action expert carries a coordinate-bound motor program that the scene selects, three predictions follow. (i) Moving an object out of its training-distribution position should break execution with the instruction unchanged, as robustness benchmarks report [Zhou et al., 2025, Fei et al., 2025]. (ii) Semantic control should require intervening on the pathway that selects the goal. Our pathway-injection asymmetry (expert injection misdirects, VLM injection stalls) is consistent with this, and failed feedforward-side semantic steering [Jin et al., 2026] is the predicted negative

result. (iii) Override should weaken as fine-tuning data grows more diverse. Swann et al. [2026] report the feature-level trend, and demonstration count alone does not move override (App. N finds it unchanged across a $10\times$ range).

Benchmark Scope

All cross-architecture results come from four simulated benchmarks. The UR10e covers $\pi_{0.5}$ on one task pair (App. O) and tests injection and coordinate binding under real sensing and actuation, while visual dominance and pathway specialization remain simulation results. Fine-tuning on a new robot reshapes internal representations, so the next step is applying the same protocol to other architectures on additional embodiments. We live this to future work, but invite the contribution to Action Atlas.

Coordinate-Frame Disambiguation

Cross-task injection and the displacement analysis (App. H.3) fit binding to absolute workspace coordinates but do not exclude scene-relative coordinates that coincide with the source-task locations. A rigid transform of the whole world in sim, would separate the two but we did not run it. Under either frame the policy commits to a trajectory tied to spatial coordinates and does not re-ground on the visible task, and the physical displaced-object test (§4.7) separates object tracking from coordinate anchoring.

Counterfactual Prompt Coverage

Our counterfactual prompt set tests simple variations (negation, null, swap). Compositional instructions ("pick up the red cup, then place it behind the blue bowl") and ambiguous multi-object scenes would provide stronger evidence for language insensitivity claims. Benchmarks like LIBERO-PRO [Zhou et al., 2025], which tests corrupted instructions and environmental perturbations, offer a stricter evaluation protocol for language grounding.

Cross-Task Injection Confounds

Injected activations are off-manifold, and temporal misalignment can produce invalid internal states. App. C.3 sets out the three observations that weigh against a drift reading: a norm-matched random direction at the same layers is at least as far off-manifold and gives chance-level override, displacement places trajectories closer to the source task than to the destination, and on hardware the random arm is the one that moves incoherently (App. O).

Concept Identification

SAE features are not guaranteed to be disentangled. Ablation validation finds that some identified features encode general motor primitives and lack a clean semantic reading.

C Methodology Controls

C.1 Discrete Tokenization Prevents SAE Intervention

Before adopting OpenVLA-OFT, we conducted experiments on base OpenVLA (autoregressive discrete 256-bin tokenization). Despite successful SAE training ($R^2 = 0.87$–0.96), hooking SAEs into the forward pass produced 0% task success on all but the final layer. The discrete tokenization maps activations to bins via argmax. Even small reconstruction errors shift the selected bin, with errors compounding across the 7-token autoregressive sequence. Replacing discrete tokenization with continuous L1 regression via OpenVLA-OFT enables SAE intervention at 99.2% success. Scale is fixed at 7B, so the action representation sets SAE applicability.

C.2 ACT as Non-VLM Control

ACT provides a control lacking any language pathway. Cross-task injection between TransferCube and Insertion is a no-op at this hook (cosine similarity $= 1.0$, bit-identical action arrays), so ACT gives no cross-task evidence. Grid ablation shows spatially structured representations. Masking

grid position (2,2), the primary manipulation workspace, drops success on both tasks (Tab. 2), and Gaussian noise ($\sigma = 0.1$) reduces TransferCube to 0% and Insertion to 25%. Therefore, visual pathway dominance holds for a vision-only policy.

C.3 Random-Direction Injection Control

To rule out the alternative that any large residual perturbation moves the policy to its closest in-distribution motor prior, we run a norm-matched random-direction control on $\pi_{0.5}$, `libero_goal`, expert layers L16+L17 (the same hook locations as the production cross-task condition). For each forward call t, we replace the source-task injection $\mathbf{h}_\ell^A$ with $r_t \cdot \|\mathbf{h}_\ell^A\|_2$, where $r_t \sim \mathcal{N}(0, I)$ is normalized to the unit sphere. The destination scene and source prompt are unchanged. App. L varies the magnitude of the directed injection itself.

Across n=50 episodes (10 pairs $\times$ 5 seeds): 22% source-behavior, 42% destination-behavior, 36% ambiguous, success rate 0%. Mean $\cos_{\text{src}} = -0.017$, mean $\cos_{\text{dst}} = 0.012$, both at noise level. The override rate (cos_src $>$ cos_dst with no threshold) is 50% [36.6–63.4], indistinguishable from chance.

Compared to the directed cross-task condition's 99.6% source-behavior on the same suite/layers (n=1,614), the gap is 78pp with non-overlapping CIs. The directed override therefore comes from the injected direction, and a random direction of the same norm only scatters the policy.

Replication at n=200. With the destination prompt held on its own task text (10 pairs $\times$ 10 episodes per suite, seeds from 42), the random direction gives an override rate of 48.0% on `libero_goal` and 43.0% on `libero_object`, 45.5% pooled (n=200), consistent with the 50% chance level, with mean cosine 0.01 to the source trajectory and 0.02 to the destination trajectory on both suites. Task success under the random direction is 30% on goal and 0% on object, against 95% and 92% uninjected. Equal-magnitude random directions therefore disrupt the policy without producing source-task behavior.

Off-manifold distance does not predict override. Injected source activations are off-distribution by construction. Fitting a Gaussian to expert-layer activations from natural rollouts of six `libero_goal` tasks (8,000 samples per layer) and scoring held-out natural samples against activations injected from the other four tasks, the Mahalanobis distance at L16 is 46.9 injected against 18.0 natural (medians 40.7 and 17.5), with 0.4% of injected samples inside the natural 95th percentile. At L17 it is 34.4 against 16.7 with 6.5% inside. Including the source tasks in the reference manifold shrinks the gap (L16 32.9 against 20.0, 9.1% inside; L17 24.8 against 17.9, 38.9% inside). The random direction sits at least as far off-manifold at the same norm and yields chance-level override, so distance from the natural manifold is not what produces the directed override. An off-manifold drift account predicts incoherent motion. Directed injection produces a complete source-task trajectory aimed at the source task's coordinates. On the UR10e, the norm-matched random-direction control is the condition that moves incoherently (App. O).

D Extended Methodology Details

D.1 SAE Architecture and Training

The 520+ SAEs of Tab. 7 are $\pi_{0.5}$ 72 (18 layers $\times$ 2 pathways $\times$ 2 pooling), OFT 96 (32 layers $\times$ 3 arms), X-VLA 120 (24 layers $\times$ 2 pooling $\times$ 2 environments, plus temporal), SmolVLA 146 (32 layers $\times$ 2 pathways $\times$ 2 pooling, plus 18 temporal), and GR00T 87 (32 layers $\times$ 2 pooling, plus 23 temporal). Concepts total 79 unique names, or 82 if the three names shared by the spatial and object categories count once per category.

Architecture. Following the TopK design of Gao et al. [2024], our sparse autoencoders consist of an encoder-decoder pair with tied weights. For an input activation $\mathbf{h} \in \mathbb{R}^d$ (per token; $d \in \{480, 960, 1024, 1536, 2048, 4096\}$ across the five VLAs),

$$\mathbf{z} = \mathrm{TopK}_k(\mathbf{W}_e(\mathbf{h} - \mathbf{b}_d)), \tag{3}$$

$$\hat{\mathbf{h}} = \mathbf{W}_e^\top \mathbf{z} + \mathbf{b}_d, \tag{4}$$

where $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ with expansion factor $m \in \{4d, 8d\}$, and $\mathrm{TopK}_k(\mathbf{u})$ retains the $k = 64$ largest pre-activations and zeros the remainder. Tied decoder weights ($\mathbf{W}_d = \mathbf{W}_e^\top$) and the pre-encoder bias subtraction follow Bricken et al. [2023], Gao et al. [2024].

Training objective. TopK enforces hard sparsity, removing the need for an L1 penalty [Cunningham et al., 2023]. We minimize reconstruction MSE plus an auxiliary AuxK term [Gao et al., 2024] that recycles dead features by reconstructing the residual $\mathbf{r} = \mathbf{h} - \hat{\mathbf{h}}$ from the top k_{aux}=512 inactive features:

$$\mathcal{L}(\mathbf{h}) \;=\; \underbrace{\|\mathbf{h} - \hat{\mathbf{h}}\|_2^2}_{\text{reconstruction}} \;+\; \alpha_{\text{aux}} \underbrace{\|\mathbf{r} - \mathbf{W}_e^\top \mathrm{TopK}_{k_{\text{aux}}}(\mathbf{z}_{\text{dead}})\|_2^2}_{\text{AuxK (dead-feature recycling)}}, \tag{5}$$

with $\alpha_{\text{aux}} = 1/32$. Decoder columns are unit-normalized after each step.

Training Hyperparameters. We train each SAE on 500,000 activation samples (approximately 10,000 forward passes $\times$ 50 action tokens) with a batch size of 4096 for 100 epochs. The learning rate is 3×10^{-4} with cosine decay. Sparsity is enforced via TopK selection with $k = 64$ active features per token. Decoder weights are tied to the encoder transpose ($\mathbf{W}_d = \mathbf{W}_e^\top$).

D.2 Concept-Based Feature Identification

Concept-associated features are selected by the frequency-weighted contrastive score $\text{score}_f = d_f \times \text{freq}_f$ defined in §3.4, with the active set taken as the top-64.

With TopK sparsity, features with high mean activation across samples do not necessarily appear in the active top-64 for individual samples, reducing their causal relevance.

Sample population for Cohen's d. Activations are first max-pooled over the 50 action-token positions per episode, yielding one activation vector per (episode, layer, feature). Cohen's d_f is computed across episodes within each concept set (n=100–200 per concept; standardized mean difference between concept-present and concept-absent episodes). Population statistics use unbiased Welch variance.

Concept specificity scores in Tab. 14(a). The dimensionless score $\text{score}_f = d_f \times \text{freq}_f$ is bounded above by $|d_f|$ and is at most a few units of effect size in our data. The values reported in parentheses in Tab. 14(a) (e.g., 133k, 412k) are raw cumulative activation magnitudes per concept, computed as $\sum_{(\text{ep},t)\in\mathcal{C}} \mathrm{ReLU}(z_{f,t,\text{ep}})$ over the concept-positive episode set $\mathcal{C}$ and the 50 action-token positions, summed across layers. We retain the raw-magnitude column for readability of layer rankings within a concept. The per-feature dimensionless score_f used to identify concept-associated features is reported in the supplementary feature-exploration platform.

D.3 Ablation Protocol

Feature ablation is performed by zeroing selected features in the SAE latent space:

$$\mathbf{z}_{\text{ablated}} = \mathbf{z} \odot \mathbf{b} \tag{6}$$

$$\mathbf{h}_{\text{modified}} = \mathbf{h} + \mathbf{W}_e^\top(\mathbf{z}_{\text{ablated}} - \mathbf{z}) \tag{7}$$

where $\mathbf{b} \in \{0,1\}^m$ is a binary mask (with m the SAE expansion dimension) carrying zeros at ablated feature indices. Per-token ablation applies this independently to each of the 50 action tokens. Writing the modification as a residual ($\mathbf{W}_e^\top(\mathbf{z}_{\text{ablated}} - \mathbf{z})$) preserves the unmodeled component of the activation and intervenes only on what the SAE explains.

Steering operator. The ablation operator generalizes to a continuous steering operator that modulates a target set of features $\mathcal{F}$ by a scalar coefficient α:

$$\mathbf{z}_{\text{steered},f} \;=\; \begin{cases} \alpha\,\mathbf{z}_f & f \in \mathcal{F}, \\ \mathbf{z}_f & \text{otherwise}, \end{cases} \qquad \mathbf{h}_{\text{steered}} \;=\; \mathbf{h} + \mathbf{W}_e^\top(\mathbf{z}_{\text{steered}} - \mathbf{z}). \tag{8}$$

$\alpha = 0$ recovers ablation, $\alpha = 1$ is identity, $\alpha \in (0,1)$ dampens, $\alpha < 0$ negates, and $\alpha > 1$ amplifies. Because TopK enforces hard sparsity, $\alpha\,\mathbf{z}_f = 0$ whenever feature f is inactive in the top-k set. This

Table 6: Architectures under study. PG = PaliGemma, Exp. = Expert, Flor. = Florence, D+E+V = DiT + Eagle + VL-SA, LIB. = LIBERO, MW = MetaWorld, SimpE. = SimplerEnv. Details in App. D.5.

Model	Params	Layers	Dim	Action Gen.	Pathway	Bench.
$\pi_{0.5}$ [Physical Intelligence, 2025]	3B	18	1024	Flow (10 steps)	Dual (PG + Exp.)	LIBERO, UR10e
OFT [Kim et al., 2025]	7B	32	4096	Cont. L1 regr.	Single (Llama-2)	LIBERO
X-VLA [Zheng et al., 2026]	∼1B	24	1024	Flow matching	Single (Flor.-2)	LIB., SimpE.
SmolVLA [Shukor et al., 2025]	450M	32	960/480	Flow matching	Dual (VLM + Exp.)	LIB., MW
GR00T [Bjorck et al., 2025]	3B	32	varies	Diff./flow	Triple (D+E+V)	LIBERO
ACT [Zhao et al., 2023]	80M	–	–	CVAE	Enc.-Dec.	ALOHA

multiplicative operator therefore modulates only currently-active concepts and cannot inject features absent from the active set. Additive steering ($\mathbf{z}_f \leftarrow \mathbf{z}_f + \alpha\,\mathbf{v}_f$ for a steering vector $\mathbf{v}_f$) injects concepts outside the active set, and App. G applies it in the VLM pathway. The cross-architecture sensitivity Fig. 7 reports ablation ($\alpha = 0$).

D.4 Intervention-Hook Protocol

Activation interventions hook one of two locations: the full *layer output* (residual stream after attention + MLP + residual addition) or, where the layer wrapper is bypassed, the *MLP output* that writes into the residual stream. Some VLAs bypass the standard DecoderLayer in their inference path, so hooking the layer wrapper does not modify what the policy consumes.

Per-architecture hook placement. Every intervention hooks the layer output (full residual stream): cross-task injection (Eq. (1)), layer zeroing, SAE training, SAE reconstruction, and SAE feature ablation and steering (Eqs. (7) and (8)), so each SAE operates on the activation it was trained on. The exception is SmolVLA, whose interleaved VLM-expert loop calls each layer's submodules directly and never runs the wrapping `DecoderLayer.forward`, so we hook `layer.mlp.forward`, the MLP write into the residual stream, for all of the above. The goal-steering edits of App. G reach SmolVLA's residual stream directly. They read the post-attention residual through a pre-hook on the layer's post-attention norm, add the MLP output, and write back through the MLP output. Hook visibility was verified by cosine identity ($\cos = 1.0$ between hooked and unhooked baselines on no-op interventions).

Reading the full-layer interventions. The OFT 14–15% recovery under null prompt plus baseline injection in Tab. 2, and the X-VLA and GR00T 0% under single-layer zeroing, replace the full layer output, so they are upper bounds on layer importance under residual-stream patching. On SmolVLA the same interventions change only the MLP write and leave the residual passthrough intact.

Table 7: Experimental scale across six models. Episodes are executed rollouts over every experiment type a model ran. SAE counts cover the per-token, mean-pool, and temporal arms (App. D.1). Concept counts overlap across models, and the total is the union of names. ‡Large ablation sweeps kept aggregate statistics only. The released archive holds per-episode artifacts for 394,000+ episodes.

Model	Episodes	SAEs	Concepts
$\pi_{0.5}$	71,500+	72	46
OpenVLA-OFT	70,700+‡	96	45
X-VLA	74,500+	120	55
SmolVLA	58,000+	146	60
GR00T N1.5	164,700+	87	39
ACT	1,870	–	–
Total	**>420,000**	**520+**	**79**

D.5 Model Architecture

Pi0.5 Architecture

$\pi_{0.5}$ uses PaliGemma (3B parameters) as its vision-language backbone and an 18-layer Gemma transformer (1024 hidden dimension) as the action expert. The action space is 7-dimensional (dx, dy,

dz, dax, day, daz, gripper). The model generates 50 actions per forward pass through flow matching with iterative denoising.

OpenVLA-OFT Architecture (Continuous)

OpenVLA-OFT shares the Llama-2 7B backbone and Prismatic vision encoder (DINOv2 + SigLIP) with base OpenVLA and replaces discrete token prediction with continuous action regression. An MLPResNet action head, trained with L1 loss, generates all 7 action dimensions in a single forward pass with 8-step action chunking (56 action tokens total). Optimized Fine-Tuning (OFT) uses LoRA adapters for parameter-efficient adaptation while preserving the pretrained representation geometry.

X-VLA Architecture

X-VLA (0.9B parameters) encodes the main camera view and the instruction with Florence-Large and generates actions with a 24-layer transformer (1024 hidden dimension) trained by flow matching. Learnable soft prompts, one set per data source, condition the shared transformer on the embodiment, and domain-specific output projections map its tokens to action chunks.

SmolVLA Architecture

SmolVLA (450M parameters) pairs a SmolVLM2 vision-language model (960 hidden dimension) with a flow-matching action expert at half that width (480). The LIBERO checkpoint we use keeps all 32 VLM layers and a 32-layer expert. The two run layer by layer: expert layers alternate cross-attention to the VLM's keys and values with causal self-attention over the action tokens. The expert generates 50-action chunks with 10 denoising steps.

GR00T N1.5 Architecture

GR00T N1.5 (3B parameters) has three pathways. A frozen Eagle vision-language model supplies the output of its 12th LLM layer, 4 vision-language self-attention (VL-SA) layers process those tokens, and a 16-layer diffusion transformer (DiT) generates actions. DiT blocks alternate cross-attention to the vision-language tokens with self-attention over state and action tokens, and produce 16-step action chunks with 4 flow-matching steps.

ACT Architecture

ACT uses a ResNet-18 vision encoder (pretrained on ImageNet) feeding into a Transformer Encoder-Decoder with a CVAE latent space ($\beta = 10$, latent dimension 32). The encoder processes multi-view camera observations, and the decoder generates action chunks of 100 timesteps in 14-DOF joint space (7 per arm for bimanual manipulation).

D.6 Benchmarks

LIBERO [Liu et al., 2023] comprises four suites of 10 tasks each: **Goal** (different goals in a shared scene), **Object** (pick-and-place with varied objects), **Spatial** (spatial reasoning and relational placement), and **LIBERO-10** (diverse tasks spanning all three categories). MetaWorld [Yu et al., 2020] provides 50 tabletop manipulation tasks grouped by difficulty (easy, medium, hard, very hard). SimplerEnv [Li et al., 2024b] evaluates sim-to-real transfer on Google Robot and WidowX embodiments. ALOHA [Zhao et al., 2023] tests bimanual manipulation with the ACT policy.

D.7 Compute Requirements

Experiments were conducted on an 8×A100-SXM4-80GB cluster for $\pi_{0.5}$, OpenVLA-OFT, large-scale SAE training, concept ablation, and cross-model analysis, an NVIDIA RTX 5090 (32GB) for GR00T N1.5 experiments, and two NVIDIA RTX 4090s (24GB) for SmolVLA and X-VLA experiments. Total experiment data exceeds 7.1 TB, including 4.3 TB of activation recordings, 152 GB of SAE checkpoints, and 420,000+ rollout episodes. The full study used approximately 10,000 A100 GPU-hours.

D.8 Evaluation Protocol

Episode counts per condition are stated with each experiment (20 to 50 per task for the main results, 5 per task for the earliest sweeps, and 3 per task for the OFT goal-steering screen). Episodes run for a maximum of 300 steps.

E Cross-Architecture Synthesis

Figure 4 condenses the cross-architecture results of §4.2–§4.6 into one view across six axes:

- **Baseline success:** the suite mean of the baseline rows in Tab. 2.
- **Visual override:** destination success lost under cross-task injection, 100 − Dst. SR in Tab. 3.
- **Language tolerance:** null-prompt over correct-prompt success, averaged over the LIBERO rows of Tab. 4 (100 means policy ignores prompt).
- **SAE fidelity:** best-arm rollout success as a fraction of baseline (Tab. 12).
- **Cross-task transfer:** the tie-inclusive override rate (Tab. 10). SmolVLA's value comes from Tab. 3, since its tie fraction is 0%.
- **Ablation tolerance:** the zero-effect rate of single-feature ablation: $\pi_{0.5}$ 54%, OFT 92%, X-VLA 82%, SmolVLA 28%, GR00T 59% (Fig. 7).

Per-architecture detail is in App. K, and the SAE pooling results are in App. H.

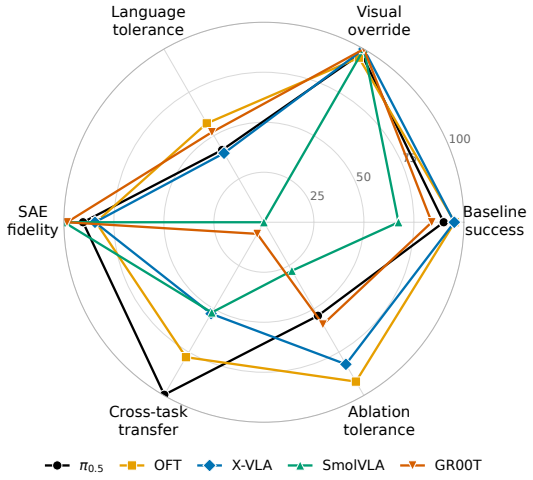


Figure 4: **Cross-model summary.** Capability radar across the five language-capable VLAs. Each axis maps a table value linearly onto 0–100, with axes defined in App. E.

The pathway-specialization claim of §4.5 rests on three architecture-specific metrics, since no single metric applies to all three multi-pathway models.

F Component-Level Analysis

This section traces information flow below the layer level: individual activation dimensions, FFN neurons, and layer-to-layer representational similarity. We analyze $\pi_{0.5}$ in depth (FFN weight projection and LDA on the 1024-dim expert space), then validate across OFT (32-layer neuron-level concept mapping, 80 concepts × 11,008 neurons per layer), SmolVLA (FFN neuron contrastive identification on both expert and VLM pathways), and GR00T (per-layer-type SAE feature utilization across DiT, Eagle, and VL-SA).

F.1 Goal Encoding Dimensions

Linear discriminant analysis (LDA) on $\pi_{0.5}$ expert activations at layer 17 identifies three components that separate four goal-suite tasks with 71.9% cross-validated accuracy (n=114, 5-fold). The first component captures 45.8% of discriminant variance, the second 35.2%, and the third 19.0%. The top 20 discriminant dimensions (417, 909, 934, 649, 147, 219, 708, 297, 155, 545, ...) define a low-dimensional subspace sufficient for goal identification within a 1024-dimensional activation space.

Goal information emerges at different rates across pathways. PaliGemma goal classification accuracy rises from 56.4% at layer 0 to 76.4% at layer 13, then declines to 68.9% at layer 17. Expert accuracy is flatter: 64.0% at layer 0, peaking at 66.4% (layer 9), settling at 62.6% by layer 17. PaliGemma's steeper emergence profile is consistent with its role as the goal encoder. The expert pathway maintains moderate goal awareness throughout but does not refine it.

Individual activation dimensions participate in both goal encoding and action generation. Dimension 62 most strongly modulates the x-component of action output (score 0.074), dimension 618 modulates y (0.071), and dimension 14 modulates z (0.066). The overlap between goal-discriminant dimensions (417, 909, 934) and action-modulating dimensions (62, 618, 14) is minimal, consistent with separable subspaces for goal identity and motor execution (tested below).

Subspace separability via subspace injection. We test separability by injecting only the goal-discriminant subspace (20 of 1024 dimensions from LDA) from task A into task B's forward pass at layer 17. Across five task pairs on `libero_goal`, full injection (all 1024 dimensions) causes complete task failure (0% success on 4/5 pairs, down from 100% baseline). Goal-subspace injection (20 dimensions, 2% of the activation space) preserves task success: 100% on three pairs, 67% on a fourth where the source and destination share the OPEN motor primitive. Action-subspace injection (15 dimensions encoding x/y/z action components) shows the same pattern. Replacing 2% of the activation space from a different task does not disrupt the destination task's motor execution, evidence for separability of goal identity from motor execution within the same layer, and stops short of direct causal control over goal selection. Additive steering along goal directions in the VLM pathway is tested in App. G.

F.2 FFN Motor Primitive Neurons

Projecting PaliGemma FFN weight vectors onto a token-association vocabulary identifies neurons that encode specific motor primitives. At layer 17, 6,606 of 16,384 neurons (40.3%) associate with motion verbs (pick, place, move, push, pull), 2,907 (17.7%) with object tokens, 2,989 (18.2%) with gripper state, 2,223 (13.6%) with direction, and 1,151 (7.0%) with spatial relations.

The distribution shifts across layers. Motion verb neurons increase from 4,231 at layer 0 (25.8%) to 6,606 at layer 17 (40.3%), a 56% increase. Gripper neurons triple from 998 (6.1%) to 2,989 (18.2%). Object neurons decrease from 3,660 (22.3%) to 2,907 (17.7%), while spatial neurons decline from 1,655 (10.1%) to 1,151 (7.0%). This progression from object/spatial representation in early layers to motor/gripper representation in late layers traces the computational transformation from scene understanding to action generation within a single pathway.

OFT's 32-layer Llama-2 backbone shows a different pattern. Granular concept mapping (80 concepts per layer, 11,008 neurons) shows that spatial relation neurons (`spatial_in`: 5,483→7,245; `spatial_on`: 4,724→6,727) dominate at all depths and grow monotonically from L0 to L31, while object neurons remain sparse (e.g., `obj_can`: 814→1,175). OFT's 4096-dim representation distributes spatial information broadly across layers, consistent with its resilience to concept ablation (92% zero effect). GR00T's SAE feature analysis shows a layer-type gradient in feature utilization, with later DiT layers using more features. DiT L0 retains only 3,654 of 12,288 features (70% dead), rising to 9,317 alive at DiT L15, while Eagle L0 has 6,730 alive of 16,384 and VL-SA L0 has 11,585 alive.

F.3 Layer Similarity via CKA

Linear Centered Kernel Alignment (CKA) between $\pi_{0.5}$ expert layers 5 and 11 on LIBERO-Goal, computed on paired samples (the same rollout steps and token positions in both layers), is 0.79. A

shuffled-row control gives 0.02. The expert therefore carries a shared residual representation that later layers refine, consistent with injected activations persisting through the remaining layers.

OFT's 32-layer Llama-2 backbone shows the same refinement pattern from the probing side: R^2 for episode-length prediction rises from 0.845 (L0) to 0.941 (L24) before settling at 0.915 (L31), while task classification accuracy saturates at 97.7–100% by L8. Combined with the FFN analysis above, these findings indicate that layers in both architectures refine a common representation across depth.

F.4 Where Spatial Binding Forms During Fine-Tuning

Comparing backbone activations between base OpenVLA (fine-tuned with discrete action tokens) and OFT (fine-tuned with continuous L1 regression) on the same LIBERO tasks localizes the representational changes induced by fine-tuning. We feed identical images through both models and measure per-layer cosine similarity of mean activations (n=50 images per suite, 4 suites).

Across all four suites, divergence grows with depth. Layers L1–L15 remain near-identical (cosine similarity >0.999), L16–L28 drift from 0.997 to 0.980, and the final layer L31 drops to 0.917–0.938. Layer 0 dips slightly (0.970–0.989) from differences in input embedding.

The long-horizon `libero_10` suite diverges most at L31 (0.917), and the three single-concept suites least (0.935–0.938).

Fine-tuning therefore reshapes the final third of the backbone and leaves early and middle layers intact. The coordinate-bound programs of §4.3 sit in the layers that change most.

Four OFT models fine-tuned on different suites (6 pairs $\times$ 32 layers, n=30 images each) differ much less at L31 (0.973–0.992) and not at all in the middle (L16: 0.998–0.999). The action head (discrete against continuous) reshapes late layers more than the choice of fine-tuning suite.

G Goal Steering in the VLM Pathway

§4.5 places goal selection in the VLM pathway. Here we test that claim causally by asking whether an edit there, with no instruction present, makes the policy complete a chosen task in a scene that holds several possible goals.

Protocol. The instruction is replaced by a filler string of function words with the same token count, so every other prompt token keeps its position. We record the VLM-pathway residual stream under the true instruction and under the filler, three rollouts each, and edit only the instruction positions (for causal backbones, the instruction and the tokens after it). We compare three edits. (i) A *patch* replaces the edited positions with their mean under the true instruction. (ii) A *per-position gap* adds, at each position, the mean difference between true instruction and filler. (iii) A *single direction* adds one unit vector, the gap averaged over positions, scaled to a fraction ρ of each token's norm. Controls are a random direction of matched norm and the gap computed for a different task. The filler and the true instruction with no edit set the floor and the ceiling.

Result. One VLM-pathway edit restores goal selection on $\pi_{0.5}$, SmolVLA, X-VLA, and GR00T, and random or wrong-task edits of the same size do not (Tab. 8). OpenVLA-OFT does not respond. On $\pi_{0.5}$ a single direction at PaliGemma layer 2 completes 113/150 episodes against 6/150 for a random direction, and nine of ten tasks respond. Edits at layers 2 and 5 succeed where a pilot at layers 9 and 13 failed, so the instruction is read early in the VLM stack.

The form of the signal depends on the architecture. $\pi_{0.5}$ carries the goal as one direction shared across instruction positions. SmolVLA carries it per position, so one averaged direction fails there at every magnitude. X-VLA's bidirectional Florence encoder spreads the instruction into the image tokens, so the edit has to cover the whole Florence block. GR00T responds to the per-position gap on its Eagle backbone at its ceiling. OpenVLA-OFT runs image and instruction through one Llama stack with bidirectional attention, and the per-position gap over the whole sequence at layer 4 or 12 completes 2/24 episodes on the eight tasks without a filler floor, against 24/24 for the true instruction.

Pooling and SAE features. The same protocol tests whether SAE reconstructions keep the steerable signal (Tab. 9). On X-VLA and GR00T, per-token and mean-pooled SAEs exist at the edited layers,

Table 8: Goal steering with the instruction replaced by a length-matched filler, as successes over episodes for the chosen task. $\pi_{0.5}$, GR00T (Eagle L6) and OFT (Llama L12, public checkpoint, official harness) run 10 `libero_goal` tasks. X-VLA runs 9, excluding one task that succeeds under the filler. SmolVLA runs 5 `libero_object` tasks.

	$\pi_{0.5}$	SmolVLA	X-VLA	GR00T	OFT
Filler, no edit	12/150	0/50	2/90	3/100	6/30
True instruction	139/150	46/50	86/90	91/100	30/30
Patch (instruction states)	137/150	44/50	–	–	–
Per-position gap	–	26/50	50/90	93/100	8/30
Single direction	113/150	0/150	–	–	–
Random, matched norm	6/150	0/150	3/90	10/100	6/30
Other task's edit	2/150	1/150	0/90	0/100	0/30

and we add the per-position gap after each one: the per-token SAE reconstructs each position, and the mean-pooled SAE reconstructs the span mean, as it was trained to. On X-VLA the per-token reconstruction steers as well as the raw gap (54/90 against 50/90 at layer 0, Fisher p=0.65), and the mean-pooled reconstruction falls to the random floor (2/90, p=7.5×10^{-19} against per-token). Pooling the raw gap without any SAE already removes most of the effect. GR00T repeats the pattern. §4.6 finds that X-VLA and the GR00T action head tolerate mean-pooling for rollout reconstruction, and the instruction signal in their language pathway still needs the per-token SAE, so pooling preference also depends on which signal the SAE must keep. On $\pi_{0.5}$, a direction built from the top 32 features of the per-token PaliGemma SAE recovers 64/150 at layer 5 against 74/150 for the dense direction. SmolVLA's SAEs are trained on MLP outputs, which do not carry the instruction in its early layers (patching them alone gives 0/50), so they cannot host this test.

Table 9: Steering through SAE reconstructions. The per-position instruction gap is added after reconstruction by each SAE, with n=90 (X-VLA) and n=100 (GR00T) per cell.

	X-VLA L0	X-VLA L2	GR00T L2	GR00T L6
Raw gap	50	48	94	93
Per-token SAE	54	46	66	88
Raw gap, pooled over positions	14	10	9	24
Mean-pool SAE, per position	13	6	8	17
Mean-pool SAE on the pooled gap	2	3	4	10
Random, matched norm	3	0	5	10

Scope. The edit is computed from rollouts under the true instruction in the same scene, so it restores a goal the policy already executes. It supplies a goal where no instruction is present and does not test overriding a competing instruction. Success rates are in simulation, with 3 (OFT) to 15 ($\pi_{0.5}$) episodes per task for each edit and magnitude. Rows that pool magnitudes (SmolVLA single direction and controls) sum over them.

H Additional Experimental Results

Tie-inclusive override rates. Table 10 reports override without the tie-band exclusion of Eq. 2, i.e., $\Pr[\Delta_t > 0]$ over all injection steps, for every model with saved trajectories. The $\pi_{0.5}$ and OFT rows use the action-vector-cosine metric of the main text, and the X-VLA and GR00T rows use the end-effector-velocity classifier. X-VLA under action-vector cosine including ties is 39.5%. Cross-architecture comparisons should be read within one metric.

$\pi_{0.5}$ null-prompt recovery and batch agreement. The $\pi_{0.5}$ null and injection rows of Tab. 2 come from one matched batch (10 tasks $\times$ 5 seeds, n=50/suite, standard chunked action execution). Against that batch's own baselines (90.0%, 92.0% and 88.0% on goal, object and spatial), injection recovers 88%, 69% and 62% of the baseline-to-null gap (Fisher p=1.1×10^{-14}, 2.6×10^{-6} and 0.13, injection against null). The n=500 sweep in Tab. 4 (31.6% object, 5.8% goal, 55.0% spatial) agrees with it on all three suites (object and goal inside the n=50 Wilson intervals; spatial 31/50 vs. 275/500, Fisher p=0.37).

Table 10: Tie-inclusive override ($\Pr[\Delta_t{>}0]$). n counts episodes with saved trajectories, so it can be smaller than in Tab. 3. *SmolVLA's per-episode classifier has no tie band, so its tie-inclusive and conditional rates coincide.

Model	n	Tie-incl. override [95% CIs]	Tie fraction	Metric
$\pi_{0.5}$	1,968	99.6% [99.2, 99.8]	0.3%	action cosine
OFT	1,079	77.9% [75.4, 80.3]	20.9%	action cosine
X-VLA	2,907	52.8% [51.0, 54.6]	32.7%	EEF-velocity cosine
GR00T	180	6.7% [3.9, 11.3]	5.6%	EEF-velocity cosine
SmolVLA	720	52.1% [48.4, 55.7]	0.0%*	per-episode override

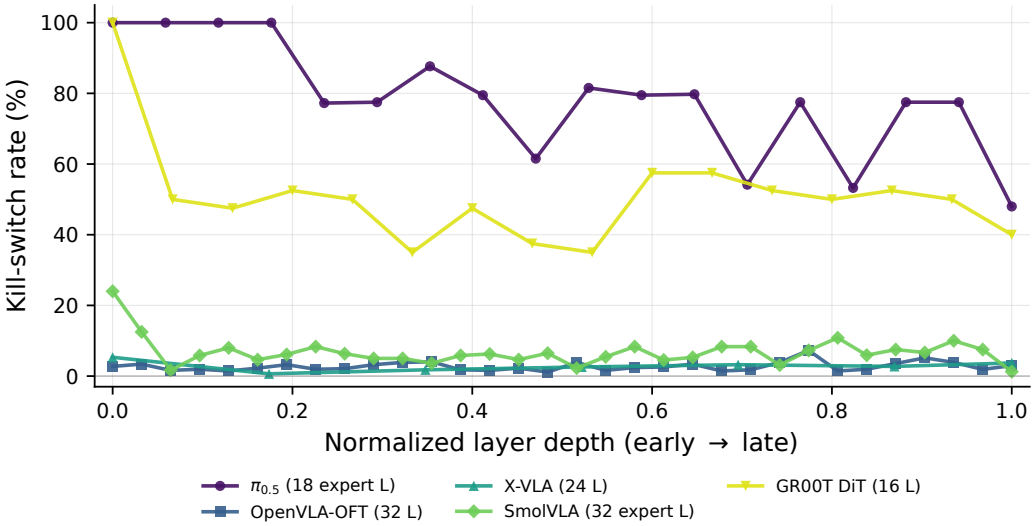


Figure 5: Per-layer kill-switch profile: the fraction of (concept, task) ablation pairs with $\Delta\text{SR} \geq 0.5$, against normalized layer depth. Pair counts are in Fig. 7.

Steering sensitivity. Sensitivity to feature steering varies across architectures. $\pi_{0.5}$ expert pathways collapse under suppression ($\Delta\text{SR} = -84$pp at $-3\times$) and GR00T DiT features under amplification (-68pp at $9\times$), while OFT (-6pp at $-3\times$) and SmolVLA (-3pp at $5\times$) tolerate the same magnitudes. We attribute the spread to the sub-millimeter precision of manipulation [Chi et al., 2023], error accumulation across the 50-token action chunk [Bengio et al., 2015], and the lack of closed-loop correction in behavior-cloned policies [Ross et al., 2011]. Phase-specific steering on $\pi_{0.5}$ localizes sensitivity to the transport phase ($p = 0.013$, two-proportion z-test).

H.1 Per-Token vs Mean-Pooled SAE Reconstruction

Fig. 6 reports the 18-layer $\times$ 2-pathway $\times$ 2-pooling-mode rollout validation on $\pi_{0.5}$/`libero_object` (n=20 episodes per (layer, pooling) cell; 1,600 episodes in total). The two SAE families are trained on success-only baseline-rollout activations under identical hyperparameters (k=64, 8$\times$ expansion, 30 epochs), differing only in whether the latent code is computed per token or after mean-pooling and broadcast back to every position.

Cross-model SAE reconstruction quality is in Fig. 12. Mean-pool collapses the $\pi_{0.5}$ expert at every layer. Success drops from 93.8% baseline to 8.9% averaged across all 18 layers. Per-token preservation holds at every expert layer (range 55–100%, 18-layer mean 83.6%). The PaliGemma pathway shows a late-stack reversal. Mean-pooled SAEs at L14–L17 preserve 60–85% success while earlier layers collapse (L0–L12 at 0%, L13 at 5%), consistent with PaliGemma's role as goal/scene encoder whose late-layer activations are position-invariant and therefore tolerate pooling. A LIBERO-10 task-by-task comparison on $\pi_{0.5}$ expert (n=250 per task) reaches the same conclusion in stronger form, with 1,748/2,500 success under per-token SAE (69.9%) vs. 9/2,500 under mean-pool (0.4%).

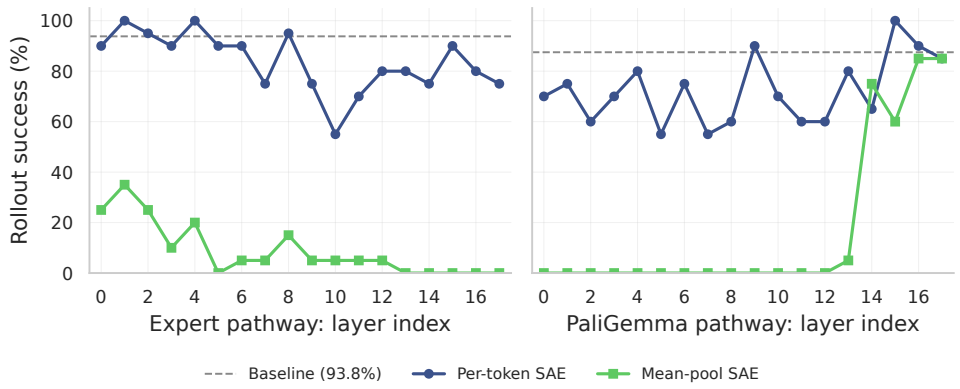


Figure 6: Per-layer rollout success through per-token and mean-pool SAEs on $\pi_{0.5}$ `libero_object` (n=20/cell). Dashed lines: per-pathway baselines (expert 93.8%, PaliGemma 87.5%).

H.2 Temporal-Contrastive SAE Pooling (T-SAE Ablation)

We test whether a temporal-aware SAE that preserves per-token operation but encourages adjacent codes to be similar would close the per-token versus mean-pool gap. We port the Temporal SAE (T-SAE) recipe of Bhalla et al. [2025] to VLA action tokens and evaluate it as a third pooling arm matched on activation source, TopK ($k = 64$), expansion ($8\times$), optimizer, and epoch count against the per-token and mean-pool baselines reported in Fig. 6. The loss and training settings are under Training details below.

Pooling preference is architecture-tied. T-SAE checkpoints are trained on five architectures ($\pi_{0.5}$ expert 18 layers, X-VLA 24, SmolVLA expert 32, GR00T-DiT 16, OFT 32) and validated by rollout on all five (Tab. 12). Two regimes appear. On $\pi_{0.5}$ (n=1,440 per arm, Tab. 11) and OFT (Tab. 17), mean-pooling collapses rollout success, so both models carry position-distinct phase information in the action-token residual stream. X-VLA (n=4,800 per arm), SmolVLA, and GR00T-DiT carry close to position-invariant action-token activations and tolerate mean-pooling, with SmolVLA indifferent within 0.8 pp. OFT's similarity to $\pi_{0.5}$, despite a different VLM and action head, places the divider in the residual-stream encoding. The temporal-contrastive arm degrades both regimes (mechanism below).

Table 11: $\pi_{0.5}$ expert 3-arm SAE pooling (n=1,440/arm, 18 layers $\times$ 4 suites $\times$ 20 ep). Deltas vs. Tab. 2 baseline.

Suite	Baseline	Per-token	Temporal	Mean-pool
libero_goal	93.2%	83.1% (–10.1)	76.1% (–17.1)	8.1% (–85.1)
libero_spatial	92.8%	**90.0%** (–2.8)	**88.3%** (–4.5)	1.9% (–90.9)
libero_object	87.6%	81.9% (–5.7)	74.7% (–12.9)	2.2% (–85.4)
libero_10	86.0%	70.3% (–15.7)	46.7% (–39.3)	1.1% (–84.9)
Mean (4 suites)	**89.90%**	**81.32%**	**71.46%**	**3.33%**

Mechanism. A diagnostic on layer 8 (1.5M adjacent pairs, 5K held out) measures how close the codes $z = \text{encode}(\mathbf{h})$ of adjacent timesteps sit. Per-token and mean-pool codes give $\cos(z_t, z_{t+1})$ of 0.857 and 0.852, so pooling leaves this geometry intact and its damage comes at inference. The temporal-contrastive arm raises it to 0.997, the optimum of its InfoNCE objective. Adjacent codes already share most of their active features (TopK Jaccard 0.61–0.71, against 0.12–0.18 for random pairs), so the contrastive loss only has to align their coefficients, and it makes adjacent codes nearly identical. That collapse removes the phase information the policy reads downstream.

Cross-architecture confirmation. The contrastive arm converges to $\cos_{\text{adj}} \approx 0.997$ on $\pi_{0.5}$, X-VLA, SmolVLA, and GR00T-DiT (SmolVLA 0.940, slightly under-converged) while keeping per-

token-grade R^2 (≥ 0.97). On OFT, training is data-limited ($\sim$26K pairs/layer vs. $\geq$ 100K elsewhere) and $\cos_{\text{adj}}$ rises only to 0.717. Per-architecture per-token reconstruction means and InfoNCE values are folded into Tab. 12.

Table 12: Rollout fidelity of the three SAE arms per architecture, with T-SAE training metrics under matched hyperparameters (α=1.0, k=64, 8×). **Bold**: best arm per row.

Model (pathway)	L	Baseline	Per-token	Temporal	Mean-pool	Recon (PT)	Recon (T)	InfoNCE
$\pi_{0.5}$ (expert)	18	89.9%	**81.3%**	71.5%	3.3%	0.018	0.026	5.26
X-VLA (transformer)	24	95.2%	50.6%	23.5%	**80.4%**	0.008	0.015	5.26
SmolVLA (expert)	32	67.3%	**67.2%**	35.1%	66.4%	0.002	0.108	5.35
OFT (Llama-2)	32	95.4%	**80.1%**	46.2%	7.3%	0.072	0.342	5.55
GR00T (DiT)	16	83.9%	80.0%	76.2%	**82.6%**	0.010	0.008	5.28

Limitations. α was not swept. We report the published Bhalla et al. [2025] default $\alpha = 1.0$. The conclusion is that at the published hyperparameters T-SAE degrades VLA rollout fidelity on every architecture we tested. Whether a tuned $\alpha < 1$ can encourage temporal smoothness without the latent-code collapse described under Mechanism is an open question.

Training details. We follow the T-SAE recipe of Bhalla et al. [2025] with TopK k=64 in place of its BatchTopK k=20 and without its Matryoshka decoder, so the three arms differ only in pooling and the contrastive term. The total loss adds a symmetric InfoNCE contrastive term to the per-token reconstruction MSE on adjacent latent codes:

$$\mathcal{L} \;=\; \mathcal{L}_{\text{recon}} \;+\; \alpha\,\mathcal{L}_{\text{contr}}, \qquad \mathcal{L}_{\text{contr}} \;=\; -\tfrac{1}{N}\sum_i \log \frac{\exp s(z_t^i, z_{t-1}^i)/\tau}{\sum_j \exp s(z_t^i, z_{t-1}^j)/\tau} \;+\; \text{(symmetric)}, \quad (9)$$

with $s(\cdot,\cdot)$ cosine similarity, $\alpha = 1.0$, $\tau = 1.0$. VLA activations are 3-D (T timesteps × S tokens-per-step × D). We use `cross_time` adjacency $(t, s) \leftrightarrow (t+1, s)$ as the most direct port of T-SAE's adjacent-token InfoNCE.

All three arms share the activations, the success-only filter, 8× expansion, Adam (lr $= 10^{-3}$, batch 512, 100 epochs), and per-position normalization. As in the original recipe, in-batch negatives can come from the same episode at a different timestep.

H.3 Dynamic Time Warping Analysis of Cross-Task Trajectories

The *cosine-dominance* override metric used in §4.3 (Eq. 2) compares per-step action-vector directions, which is a local geometric measure. We add a global trajectory-shape measure: Dynamic Time Warping (DTW) on end-effector position trajectories, normalized by warping-path length. For each cross-task pair (A, B) with prompt B and source A's expert activations injected at layers 16–17, we compute three DTW distances per episode: $d_{\text{src}} = \text{DTW}(\mathbf{x}^{B|A\text{-inj}}, \mathbf{x}^A)$, $d_{\text{dst}} = \text{DTW}(\mathbf{x}^{B|A\text{-inj}}, \mathbf{x}^B)$, and the inherent $d_{A,B} = \text{DTW}(\mathbf{x}^A, \mathbf{x}^B)$ as a control. We report the fraction of episodes with $d_{\text{src}} < d_{\text{dst}}$ as DTW source-dominance, paralleling the cosine-dominance override rate.

DTW corroborates the cosine result on `libero_goal` (90.7% source-dominance, n=270) and `libero_spatial` (86.4%, n=44). The 8–12pp gap between cosine and DTW source-dominance on these suites is expected because cosine measures action-vector *direction* step-wise while DTW measures trajectory *shape* globally. An injected episode that briefly aligns with the source then diverges can register as cosine-dominant but DTW-mixed.

The long-horizon `libero_10` suite shows weaker DTW agreement (45.8% source-dominance vs. 95.8% cosine-dominance, n=24). The inherent $d_{A,B}$=0.361 is large and injection-induced trajectory truncation often shortens the rollout, making the truncated trajectory artifactually closer to the shorter destination baseline by raw shape distance. Cosine, normalized per step, is insensitive to this truncation, so the `libero_10` gap reflects trajectory length and leaves the cosine result intact.

Phase-aligned injection. Step-index injection assumes the source and destination episodes are in phase. We test this by aligning source to destination steps with dynamic time warping on the end-effector trajectories (9 `libero_goal` pairs × 6 episodes, n=54 per arm, expert L16+L17).

Table 13: DTW source-dominance for cross-task injection on $\pi_{0.5}$, alongside per-step cosine source-dominance

Suite	n	DTW→src	DTW→dst	$\text{DTW}_{A,B}$	DTW src-dom. [95% CI]	Cos src-dom.
LIBERO-Goal	270	0.105±0.055	0.232±0.104	0.190	90.7% [86.7, 93.6]	98.9%
LIBERO-Spatial	44	0.121±0.091	0.160±0.089	0.194	86.4% [73.3, 93.6]	97.7%
LIBERO-10 (long)	24	0.338±0.225	0.225±0.073	0.361	45.8% [27.9, 64.9]	95.8%
Pooled	338	0.124	0.222	–	87.0% [83.0, 90.2]	98.5%

Override is 94.4% under DTW alignment against 100% under raw step index (Fisher p=0.243). Shuffling the source steps lowers it to 87.0% (p=0.013), so the test can detect a dependence on order. A budget-matched warp (88.9%) and gripper-phase alignment (83.3%) agree, and the no-injection floor is 0.0%. The DTW path is the identity on 5 of 9 pairs of this checkpoint. On the three App. N checkpoints, where alignment changes about 70% of injected steps, override is 98.1–100% under raw index, 66.7–68.5% under DTW, and 70.4–72.2% under shuffling, against floors of 1.9–5.6%. On X-VLA, DTW alignment gives 88.9% against 100% under raw step index (p=0.027) and a 0.0% floor (n=54 per arm). Most of the override survives any reordering of the source steps, and the order-dependent part is the same under phase alignment and under shuffling.

H.4 $\pi_{0.5}$ Concept-Feature Analysis

Tab. 14 reports which expert layers carry each concept and whether one feature separates the tasks that use it. Later expert layers (L14–L17) carry the strongest signal, and each top feature classifies its tasks at 100% accuracy.

Table 14: $\pi_{0.5}$ concept features. (a) The three expert layers with the largest summed activation $\sum_t \text{ReLU}(z_{f,t})$ per episode for each concept's top feature, in thousands. (b) Whether the top feature alone separates the LIBERO-10 tasks that use the concept.

(a) Layer specificity

Concept	**Best Layer**	**2nd Best**	**3rd Best**
Action concepts			
PUT	L17 (133k)	L16 (111k)	L15 (86k)
OPEN	L15 (103k)	L16 (79k)	L13 (71k)
PUSH	L17 (412k)	L13 (275k)	L14 (270k)
INTERACT	L17 (274k)	L12 (261k)	L15 (253k)
Object concepts			
BOWL	L16 (114k)	L15 (80k)	L14 (70k)
WINE_BOTTLE	L16 (128k)	L14 (116k)	L17 (104k)
STOVE	L17 (182k)	L15 (164k)	L12 (141k)

(b) Task classification on LIBERO-10

Concept	**Tasks Active**	**Tasks Inactive**	**Accuracy**
PUT	1,3,4,5,6,7,8	0,2,9	100%
OPEN	0,1	2–9	100%
INTERACT	9	0–8	100%

On $\pi_{0.5}$ libero-object, concept-aligned selection (Cohen's $d_f \times \text{freq}_f$) preserves rollout success under ablation (93% → 93%), while action-correlated selection (linear-probe x,y,z,rpy,gripper) drops success by 14 pp because the chosen feature tracks per-step motor output statistics and carries little concept identity.

Causal Necessity of Linear Probe Directions

Linear probes trained to predict action dimensions achieve 97–98% R^2. Projecting out probe directions eliminates linear decodability by construction (R^2 drops to ≈ 0). The causal test is behavioral. Rollouts under projection fail (§3.5), indicating the network itself relies on information in

these directions. Cross-architecture single-feature ablation sensitivity is reported in Fig. 7 (n=15,096 concept-task pairs across five models).

Vision Robustness

Systematic image perturbation across 6,000+ episodes reveals task-dependent visual robustness. We apply horizontal and vertical flips, rotations (90°, 180°, 270°), center crops (50%, 75%), and object-centric crops. Horizontal and vertical flips break all models tested (0% success), so spatial orientation is rigidly encoded. Rotation and crop robustness varies by task complexity. Simple pick-and-place tolerates mild perturbations while multi-step tasks fail. Object-centric cropping (centering on the manipulation target) outperforms static crops (60% vs. 0–20%), pointing to reliance on manipulation-relevant regions, with full scene context contributing less.

I Ablation Studies

I.1 Temporal Ablation: GR00T Phase-Conditioned Feature Importance

Table 15 shows GR00T N1.5 temporal ablation aggregated over the full 32-layer stack across three LIBERO suites (160 conditions). Early-window ablation (-29.7pp pooled) is nearly as severe as full-episode ablation (-30.5pp), while mid- and late-window ablation cause only -13.8 and -13.6pp. The asymmetry holds on goal and long, and object stays near ceiling in every window. Layer-type-resolved profiles show DiT layers carry the asymmetry (early-window -50pp, similar to full-episode) while Eagle LM layers are flatter (~ -15pp early). The effect scales with task complexity. `libero_long` (303-step mean) shows a -44pp early drop vs. -27pp on `libero_goal` (108-step mean). The per-layer-type decomposition is not separately tabulated here. GR00T is most sensitive during trajectory initiation, after which it tolerates feature ablation.

Table 15: GR00T N1.5 temporal ablation by LIBERO suite (160 conditions across 32 layers).

Suite	Baseline	Full	Early	Mid	Late
LIBERO-Goal	100%	73.1%	73.2%	92.7%	92.4%
LIBERO-Long	100%	53.8%	55.6%	75.3%	75.9%
LIBERO-Object	100%	93.9%	94.0%	95.0%	95.3%
Pooled (160 cond.)	**100%**	**69.5%**	**70.3%**	**86.2%**	**86.4%**

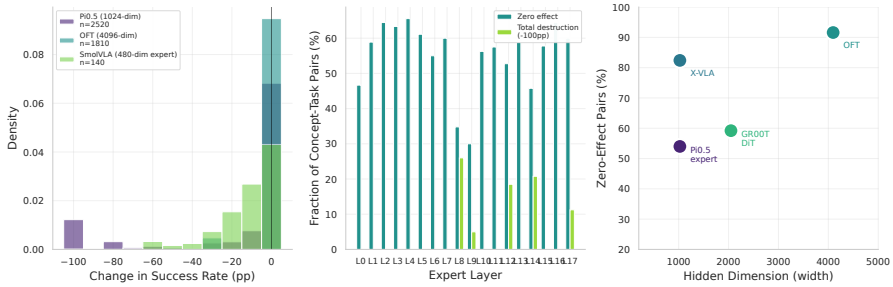
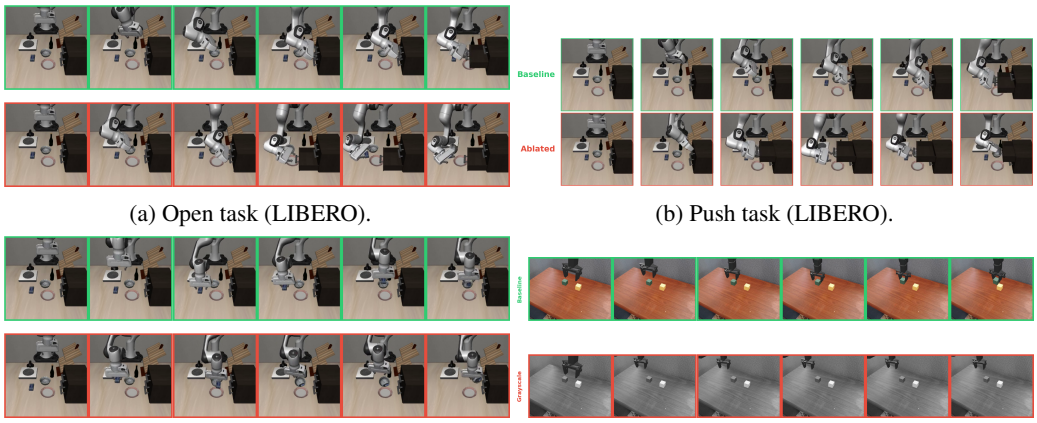


Figure 7: Single-feature ablation sensitivity. Left: change in success per (concept, task) pair for $\pi_{0.5}$ (n=2,520), OFT (1,810) and SmolVLA (140). Middle: per-layer fraction of zero-effect and total-destruction pairs on the $\pi_{0.5}$ expert. Right: zero-effect rate against hidden width for four models.

J Qualitative Results and Additional Figures

On SmolVLA and X-VLA, feature ablation and vision perturbation give binary outcomes: the robot completes the task or fails it, and failed episodes end in a partial reach. For example, SmolVLA 50% center-crop removes button-press spatial context (400-step timeout vs. 67-step baseline), and X-VLA grayscale removes color discriminability for the WidowX stack-cube task (0% success). Figure 8 shows baseline and ablated rollouts. Rollout videos are on Action Atlas (https://action-atlas.com).



(a) Open task (LIBERO). (b) Push task (LIBERO).

(c) Put task (LIBERO). (d) Grayscale perturbation (SimplerEnv WidowX).

Figure 8: Baseline (green) and ablated (red) task rollouts. (a)–(c) Feature ablation on LIBERO. (d) Grayscale input on SimplerEnv.

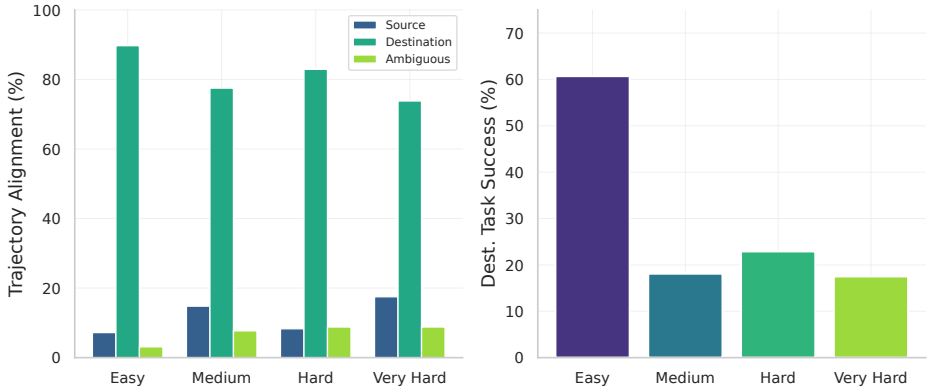


Figure 9: SmolVLA on MetaWorld by difficulty. Left: destination-behavior retention and source-task override under cross-task injection. Right: destination success under injection.

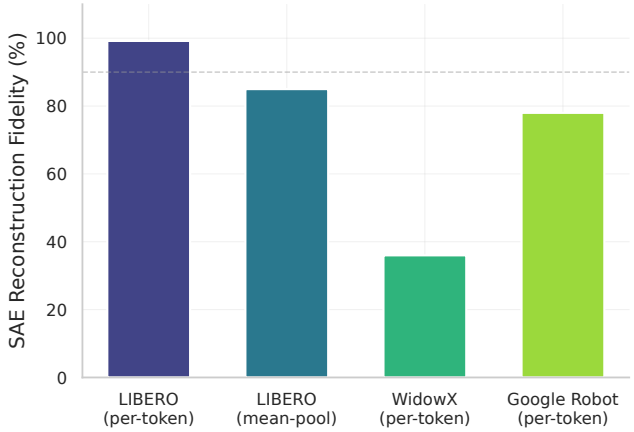


Figure 10: X-VLA SAE rollout fidelity across environments: success with the SAE reconstruction substituted at rollout time, relative to baseline.

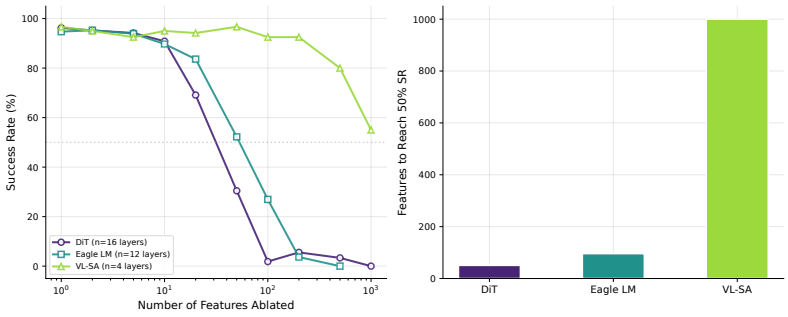


Figure 11: GR00T N1.5 success drop by layer type. DiT layers (16) drop 40–80%, Eagle LM layers (12) drop moderately, and VL-SA layers (4) are the most resilient.

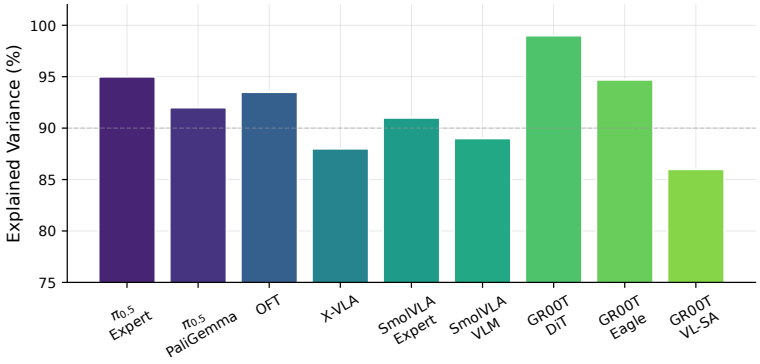


Figure 12: SAE explained variance by architecture and pathway (per-token SAEs, best layer). Dashed line: R^2=0.90.

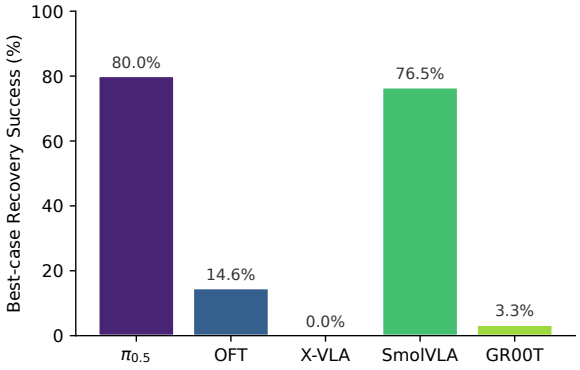


Figure 13: Best-case success under each architecture's visual-pathway intervention, from the rows of Tab. 2: PaliGemma injection under a null prompt ($\pi_{0.5}$), baseline injection under a null prompt (OFT), single-layer zeroing (X-VLA), single expert-layer zeroing on its best suite (SmolVLA), and null-source DiT injection (GR00T).

K Per-Architecture Deep Dives

Each subsection gives the architecture's distinctive failure mode and its per-suite table. Baselines match Tab. 2 except where noted, and Fig. 13 summarizes each architecture's best visual-pathway intervention. The ACT control is in App. C.2.

K.1 OpenVLA-OFT

OFT has the highest raw baseline (Tab. 2), and on the `libero_10` task-8 seed cluster where $\pi_{0.5}$ scores 0/25 it scores 23/25. Its dominant failure is structural. Zeroing any single residual-stream layer (0, 8, 16, 24, or 31) drives every suite to 0%, and injecting baseline visual-pathway activations under a null prompt recovers little (Tab. 16). OFT has the lowest null-injection recovery of the models tested. Temporal-window null injection on `libero_10` drops the nine tasks that succeed at baseline to 0% in the early and middle windows, and late nulling leaves two of them intact. Episode-length probes are slightly stronger in late layers on every suite (early/mid/late R^2: spatial 0.984/0.991/0.986, object 0.719/0.834/0.863, goal 0.859/0.896/0.933, `libero_10` 0.733/0.779/0.827).

Table 16: OFT per-suite deep dive. Zero-layer: any of layers 0/8/16/24/31 nulled under empty prompt (baseline ∼90%). Recovery: null + inject zero-baseline activations. Probe R^2: episode-length probe over 32 layers, n=149/suite.

	Null injection		Same-scene	Probe R^2 (32 L)	
Suite	**Zero-layer**	**Recovery**	**inj. SR**	**Min**	**Mean**
libero_goal	∼0%	33/234 (14%)	87.8% (n=582)	0.845	0.896
libero_object	∼0%	43/294 (15%)	77.8% (n=162)	0.608	0.813
libero_spatial	∼0%	31/216 (14%)	74.0% (n=192)	0.969	0.988
libero_10	0/10 tasks	–	71.8% (n=582)	0.692	0.780

SAE rollout per suite. Tab. 17 breaks the OFT means down by suite. The lower `libero_10` per-token mean (74.5%) is spread across layers (Fig. 14), with no single layer driving it, and all four suites decline after L20.

Table 17: OFT 3-arm SAE rollout per-suite (32 layers × 25 ep per cell). Per-token wins on every suite by 26–40pp, and mean-pool stays in single digits.

Suite	**Per-token**	**Temporal**	**Mean-pool**
libero_object	**86.6%**	54.6%	9.2%
libero_spatial	**78.2%**	51.8%	6.9%
libero_goal	**81.0%**	43.8%	8.0%
libero_10	**74.5%**	34.5%	5.1%
Mean (4 suites)	**80.1%**	46.2%	7.3%

Per-suite OFT displacement. Table 18 breaks the pooled OFT override of Tab. 3 down by suite, and source-dominant displacement is the majority outcome on every suite. Tie-inclusive override is 77.9% pooled and 73.7–81.9% per suite. Each pair runs in one injection direction, where the $\pi_{0.5}$ analysis runs both. On `libero_object` the destination task still succeeds in 19.7% of injected episodes, so there the override rate describes the episodes that left the destination task. The OFT cosines are computed from end-effector velocity and may understate displacement where steering acts in action space more than in Cartesian motion.

K.2 Cross-Model Linear Probing

Task-relevant information is linearly decodable on every architecture (Tab. 19). Expert and action pathways encode state dynamics, and VLM pathways encode goal semantics.

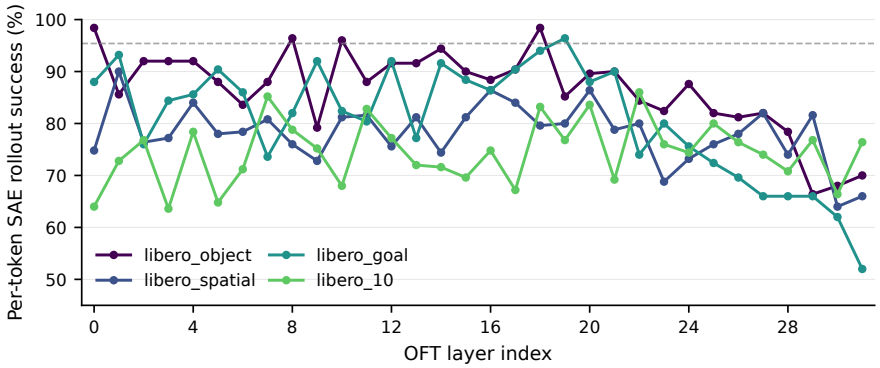


Figure 14: OFT per-token SAE rollout success by decoder layer and LIBERO suite. Dashed line: rollout baseline (95.4%).

Table 18: OFT cross-task injection per LIBERO suite, 45 pairs per suite in one direction. Each pair gives 6 episodes, one per injection layer (L0, L8, L16, L24, last, all), and one short `libero_object` episode was dropped. Override is tie-band-conditional (Eq. 2). Dest. success: destination-task success under injection.

LIBERO suite	n pairs	Episodes	Override rate (95% Wilson CI)	Dest success
`libero_goal`	45	270	100.0% (98.3–100.0%)	0.0%
`libero_object`	45	269	97.0% (93.7–98.6%)	19.7%
`libero_spatial`	45	270	90.8% (86.4–93.9%)	0.4%
`libero_10`	45	270	97.7% (94.7–99.0%)	0.0%
Pooled (all suites)	180	1,079	96.4% (94.8–97.5%)	5.0%

Table 19: Cross-model linear probing. Oracle ratio: probe R^2 over oracle R^2, the fraction of ground-truth state linearly decodable from activations. The expert/VLM oracle split is measured on SmolVLA only, and §4.5 gives the $\pi_{0.5}$ and GR00T evidence.

Model	**Probe Target**	**Metric**	**Result**
$\pi_{0.5}$ (expert)	State prediction	R^2	0.45
$\pi_{0.5}$ (PaliGemma)	Prompt classification	Accuracy	99.3%
OFT	Task identification	Accuracy	97.7–100%
OFT	Success prediction	AUC	0.97
OFT	Episode-length (mean)	R^2	0.87
SmolVLA (expert)	Oracle state ratio (h=10)	ratio	0.58
SmolVLA (VLM)	Oracle state ratio (h=10)	ratio	0.13
GR00T (all layers)	Task identification	Accuracy	100%
GR00T (all layers)	Success prediction	Accuracy	96.4%
GR00T (DiT L14)	Success prediction	Accuracy	97.7%

K.3 $\pi_{0.5}$

$\pi_{0.5}$ reaches 89.9% on the 1,000-rollout baseline (Tab. 2). Its dominant failure is coordinate-cluster brittleness. On `libero_10` task 8 one contiguous 25-seed range gives 0/25 while the neighboring ranges give 23/25 and 20/25 on the same task and policy. This is the surface form of the coordinate-bound program of §4.3, and the LIBERO-PRO position perturbations of Zhou et al. [2025] drop $\pi_{0.5}$ from 0.92–0.98 to 0.08–0.38 on all four suites.

Object identity does not predict reach target. Swapping the object noun with the scene fixed leaves the final end-effector position unchanged. One-way ANOVA of final XYZ against the named object: `libero_object` $F(10, 209)$=1.27, p=0.247, η^2=0.057 (n=220); `libero_goal` $F(9, 170)$=0.30, p=0.973, η^2=0.016 (n=180); `libero_spatial` $F(5, 54)$=2.20, p=0.067,

η^2=0.169 (n=60). None of the three reaches $p<0.05$, and the spatial test has low power (n=60). With the null-prompt result, this shows that $\pi_{0.5}$ needs an object noun to engage the task and does not use the noun to choose where to reach.

Cross-task injection per suite. Injection lowers destination success on every suite. On `libero_goal`, 91.5% falls to 0.0% under a cross-prompt without injection (n=236 per condition; PaliGemma-all 0.4%, expert L16 1.3%). On `libero_spatial` (n=34), 100% falls to 64.7% under a cross-prompt and 20.6% under the own prompt with PaliGemma-all injection. On `libero_10` (n=24), 62.5% falls to 0% under every injection condition.

K.4 X-VLA

X-VLA reaches 95.2% on the baseline and matches the $\pi_{0.5}$ injection signature (§4.3), including the `libero_10` task-8 seed cluster (0/25, with 24/25 on the neighboring range). Its failures are visual and all-or-nothing, with vision perturbations giving full success or 0% (Fig. 8d). Its null-prompt success depends on the suite, and concept ablation barely moves it (Tab. 20). Its SAEs prefer mean-pooling (80% against 51% per-token over 24 layers), so per-token SAEs lower X-VLA rollout success. The dictionaries are environment-specific (Fig. 10). At layer 0 the LIBERO SAE reconstructs LIBERO activations at normalized MSE 0.010 per-token and 0.006 mean-pool (R^2=0.99), and on WidowX activations it reaches MSE 0.66 (R^2=0.34), falling to $R^2 \leq 0$ by mid-depth.

Table 20: X-VLA per-suite breakdown. Zero any layer: any of 24 tested layers nulled. Null prompt: success under empty string. Zero eff./Δpp: concept ablation zero-effect rate and mean delta. n=4,800 counterfactual episodes and n=2,480 concept-ablation pairs.

Suite	Baseline	Zero any layer	Null prompt	Zero eff.	Δpp
Goal	98.8%	0%	10%	82.6%	−2.2
Object	98.8%	0%	60%	98.2%	−1.3
Spatial	95.6%	0%	48%	85.2%	−2.6
LIBERO-10	87.6%	0%	28%	74.7%	−2.4

K.5 SmolVLA

SmolVLA reaches 67.3% on the baseline and fails in two ways. Tasks 0 and 4 of `libero_10` sit far below the suite mean of 41.2%. On the task-8 seed cluster it scores 0/25 and recovers only to 12% and 60% on the neighboring ranges. On MetaWorld, injection leaves SmolVLA's destination behavior mostly intact (89.7% easy to 73.8% very hard) and source override low (12.4% aggregate, Fig. 9). Expert concept ablation hurts `libero_spatial` most, while VLM concept ablation hurts `libero_goal` most and helps `libero_object`, consistent with the VLM encoding goal semantics (Tab. 21).

Table 21: SmolVLA per-suite breakdown. Expert zero: success-rate range when zeroing individual expert layers. Zero eff./Δpp: expert-pathway concept ablation (n=1,696). VLM concept ablation (n=210): goal −10.7pp, object +6.4pp, spatial −8.5pp, LIBERO-10 +5.9pp.

Suite	Baseline	Expert zero	Zero eff. (exp)	Δpp (exp)
Goal	74.8%	60–83%	14.6%	−4.7
Object	87.6%	60–83%	0.0%	−3.9
Spatial	65.6%	47–77%	32.7%	−17.7
LIBERO-10	41.2%	0–33%	20.3%	+0.8

K.6 GR00T N1.5

GR00T reaches 83.9% on the baseline (Tab. 2, spatial*). Zeroing any of the 16 DiT layers drives every suite to 0%, while VL-SA layers keep 40–80% (Fig. 11), so the motor-expert pathway carries the load. Language sensitivity tracks the suite as on X-VLA, and `libero_long` has the lowest concept-ablation zero-effect rate (42.2%) and the highest destruction rate (11.0%, Tab. 22). The concept-ablation baselines in the table come from a separate batch (n=10 per task).

Checkpoints. The goal results use `Tacoin/GR00T-N1.5-3B-LIBERO-GOAL` and the `libero_10` results use `liorbenhorin-nv/groot-libero_10-64_40000`, the public checkpoints with usable baselines (alternatives reach 10% or less). *The spatial export was trained at batch size 128 for 20K steps, and the NVIDIA reference reports 97.65% at batch size 640. Community fine-tunes span 68–94%, so we report only the spatial baseline (Tab. 2, †). The qualitative findings hold on the three suites with full coverage.

Table 22: GR00T N1.5 per-suite breakdown. Zero DiT: any of 16 DiT layers nulled. CF "other": counterfactual success under non-baseline prompts (n=180/suite). Zero eff./Δpp: concept ablation across 6,500 pairs.

Suite	Baseline	Zero DiT	CF "other"	Zero eff.	Δpp
Goal	97.0%	0%	18.9%	68.8%	-5.8
Object	99.0%	0%	73.3%	69.6%	-9.9
Long	75.0%	0%	61.7%	42.2%	-5.3

K.7 Operational Summary: Failure-Attribution Rules

The failure modes above give three diagnostics. A confident reach to the wrong location points to corruption in the expert pathway (§4.3), and a stall with no committed motion points to corruption in the VLM pathway (App. F.1). Both apply to the models with separate pathways ($\pi_{0.5}$, SmolVLA, GR00T). A collapse on a contiguous range of seeds, with the neighboring ranges intact, points to brittleness around one layout of object coordinates and away from a model-wide regression. It appears where absolute coordinates dominate the action distribution ($\pi_{0.5}$, X-VLA, SmolVLA), and OFT succeeds on the same seeds.

L Dose-Response of Expert Injection

The cross-task result of Tab. 3 replaces the expert residual wholesale (α=1), so it cannot tell a program that switches on at full magnitude from a bias that grows with magnitude. A switch predicts destination behavior at every partial dose and a step at α=1, and a bias predicts a graded rise. On $\pi_{0.5}$ `libero_goal` we blend the expert L16/L17 output as $(1-\alpha)\,h_{\text{live}} + \alpha\,h_{\text{src}}$, with h_{src} the recorded source activation of Eq. 1 from one uninjected source rollout and the destination prompt held fixed. Seven task pairs run 5 episodes at each $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$. An episode is source- or destination-classified when its action-trajectory cosine to the source rollout exceeds or trails its cosine to an uninjected destination rollout by more than τ=0.05 (Eq. 2).

Table 23: Dose-response of expert injection on $\pi_{0.5}$ `libero_goal`, 7 task pairs $\times$ 5 episodes per dose. Unanimous: pairs with all 5 episodes source-classified. Cosines are episode means.

α	Source-classified	Unanimous pairs	Dst. success	$\overline{\cos}_{\text{src}}$	$\overline{\cos}_{\text{dst}}$
0.00	0/35 = 0.0%	0/7	34/35 = 97.1%	0.360	0.904
0.25	4/35 = 11.4%	0/7	19/35 = 54.3%	0.475	0.782
0.50	34/35 = 97.1%	6/7	3/35 = 8.6%	0.705	0.312
0.75	35/35 = 100%	7/7	5/35 = 14.3%	0.604	0.154
1.00	35/35 = 100%	7/7	4/35 = 11.4%	0.551	0.147

Source-classified episodes rise from 0/35 at α=0 to 4/35 at 0.25 and 34/35 at 0.5, then saturate, and the mean cosines cross in the same interval (Tab. 23). The 0.5 dose alone rules out a step confined to α=1, so we read the effect as graded biasing of the expert trajectory. A small dose first degrades the commanded task, with destination success at 19/35 for α=0.25, and redirection follows at 0.5. The 35 episodes per dose form 7 clusters that share a source rollout. At the cluster level no pair is unanimously source below 0.5, 6/7 are unanimous at 0.5 and 7/7 at 0.75 and 1.0, and the four source episodes at 0.25 come from two pairs. The scope is one checkpoint, one suite, and one layer pair, with sampling noise unmatched as in the other simulation runs (App. O).

M SAE Sparsity Sweep

The pooling comparison of §4.6 uses k=64. We repeat the per-token and mean-pool rollout on $\pi_{0.5}$ at $k \in \{32, 128\}$, expert layers L5 and L11, on all four LIBERO suites (n=30 per cell), under the replacement write-back of Fig. 6. The two `libero_10` arms have separate baseline runs.

Table 24: Per-token and mean-pool SAE rollout success on $\pi_{0.5}$ across sparsity k, n=30 per cell, against the uninjected baseline of the same run. Per-token and mean-pool Wilson 95% intervals are disjoint in every cell. Mean-pool uses the replacement write-back.

Suite	Layer	k	Baseline	Per-token	Mean-pool (replacement)
`libero_goal`	L5	32	28/30 = 93.3%	19/30 = 63.3%	0/30 = 0.0%
`libero_goal`	L5	128	28/30 = 93.3%	29/30 = 96.7%	4/30 = 13.3%
`libero_goal`	L11	32	28/30 = 93.3%	15/30 = 50.0%	2/30 = 6.7%
`libero_goal`	L11	128	28/30 = 93.3%	28/30 = 93.3%	3/30 = 10.0%
`libero_object`	L5	32	26/30 = 86.7%	19/30 = 63.3%	0/30 = 0.0%
`libero_object`	L5	128	26/30 = 86.7%	24/30 = 80.0%	1/30 = 3.3%
`libero_object`	L11	32	26/30 = 86.7%	13/30 = 43.3%	0/30 = 0.0%
`libero_object`	L11	128	26/30 = 86.7%	26/30 = 86.7%	0/30 = 0.0%
`libero_spatial`	L5	32	24/30 = 80.0%	16/30 = 53.3%	1/30 = 3.3%
`libero_spatial`	L5	128	24/30 = 80.0%	26/30 = 86.7%	2/30 = 6.7%
`libero_spatial`	L11	32	24/30 = 80.0%	8/30 = 26.7%	0/30 = 0.0%
`libero_spatial`	L11	128	24/30 = 80.0%	28/30 = 93.3%	0/30 = 0.0%
`libero_10`	L5	32	27/30 = 90.0%	11/30 = 36.7%	0/30 = 0.0%
`libero_10`	L5	128	25/30 = 83.3%	23/30 = 76.7%	0/30 = 0.0%
`libero_10`	L11	32	27/30 = 90.0%	8/30 = 26.7%	0/30 = 0.0%
`libero_10`	L11	128	25/30 = 83.3%	21/30 = 70.0%	0/30 = 0.0%

Per-token success is above mean-pool in all 16 cells, by at least 26.7 pp. Pooled over cells, per-token is 314/480 = 65.4% and mean-pool 13/480 = 2.7%. Larger k raises per-token success in every suite and layer and leaves the ordering unchanged.

Seed and corpus axes. At k=64 on `libero_goal`, three initialization seeds give per-token 80.0% against mean-pool 20.0% at L5, and 86.7% against 6.7% at L11 (n=30 per layer). The ordering holds for every seed, and the single-seed intervals at n=10 overlap for two of the six seed-layer pairs. Training on 25%, 50%, and 100% of the corpus (n=50 per cell, baseline 96.0%) gives per-token 78.0–92.0% and mean-pool 0.0–26.0%, with disjoint intervals in all six cells.

Scope. The mean-pool column holds for the replacement write-back. A residual write-back, which adds only the SAE's edit to the original activation, recovers mean-pool success on OFT (0.0% to 73.3%), GR00T (2.0% to 82.0%), and X-VLA (3.3% to 100.0%) at single matched cells. We have no $\pi_{0.5}$ mean-pool result under the residual write-back. The per-token column is the same under both write-backs, which coincide without pooling.

N Fine-Tuning Data Scale

Whether coordinate binding is an artifact of the low-data regime of task-specific fine-tuning is testable by retraining at reduced data and repeating the injection protocol. We fine-tuned $\pi_{0.5}$ (`pi05_libero_base`) on `libero_goal` at three fractions of the training set with an identical recipe (LoRA rank 32, α=64, dropout 0.05, 6,000 steps, batch 32, learning rate 10^{-4}, warmup 200, seed 42): 10% (168 episodes), 30% (512), and 100% (1,693). Each checkpoint then ran the same four-condition cross-task injection at expert L16/L17 over 9 task pairs × 6 seeds (n=54/condition). Override is the tie-inclusive rate of Tab. 10, $\Pr[\Delta_t > 0]$.

A tenfold reduction in fine-tuning data leaves the override at 54/54 and the floor at 1/54, with baseline success flat and all 9 task pairs unanimous at every fraction, so the effect is present at the smallest fraction that still trains a competent policy. Under the tie-conditional rule of Eq. 2 (τ=0.05) the 10% arm is 53/54, the one difference being a single episode in the tie band.

Table 25: Cross-task injection on $\pi_{0.5}$ as a function of fine-tuning data. Baseline is single-task success over 10 tasks $\times$ 10 episodes. Override, no-injection floor, and shuffled control are n=54 each.

Data fraction	Baseline	Directed override	No injection	Shuffled
10% (168 ep.)	97/100	54/54	1/54 = 1.9%	38/54 = 70.4%
30% (512 ep.)	97/100	54/54	1/54 = 1.9%	45/54 = 83.3%
100% (1,693 ep.)	98/100	54/54	1/54 = 1.9%	40/54 = 74.1%

O Physical Replication on a UR10e

This appendix reports a physical campaign that tests the two claims a real rig can falsify directly: that wholesale expert replay redirects which task the policy performs, and that the transplanted program is anchored to workspace coordinates.

Setup. We fully fine-tuned $\pi_{0.5}$ (20k steps) on a Universal Robots UR10e over seven tabletop tasks from 335 teleoperated episodes (60,416 frames at 15 Hz) in the DROID observation format, with two exterior cameras and a wrist camera at 320×180. Fig. 3 uses the exterior camera facing the robot. Baseline success is 76.2% (160/210, 30 trials per task, three observers), from 66.7% to 86.7% per task.

Intervention. The procedure is the simulation procedure unchanged. Captured expert activations at L16 and L17 replace the live ones at each inference call (α=1) under the fixed destination prompt.

Flow matching requires matched sampling noise. Wholesale replacement of the expert residual fixes the velocity field, so the denoiser no longer cancels its own initial noise and returns $x_{\text{final}} = a_{\text{source}} + (\epsilon_{\text{dest}} - \epsilon_{\text{source}})$. On the arm this offset showed up as weak override and an over-scaled gripper, so the physical injector reuses the source's per-call initial noise. In simulation, replaying the source's L16/L17 activations with the source's x_T reproduces the source action exactly, and a fresh x_T leaves an error of 2.18 and 3.73 over two draws (action norm 1.03). The simulation runs behind Tabs. 3 and 25 leave this term uncontrolled. Their override rate compares action directions over full episodes and still reaches 54/54, while the arm executes the raw values. Activation-transfer studies on flow-matching or diffusion heads need to control this term, which discrete and autoregressive heads do not have.

Bidirectional override. Each direction ran directed injection, a matched-scene no-injection floor, and a norm-matched random direction at the same layers and magnitude, with rollouts scored as source, destination, or neither (Tab. 5). Directed intervals exclude both control intervals in both directions, so the effect comes from the content of the injected activations. The 18/26 direction is limited by grasp failure on the compliant plushie. The arm reached for the injected target first in every trial, and the non-source trials diverted only after a failed grasp.

Displaced-object test. Coordinate anchoring predicts that the arm goes to the recorded position of a moved object, vision tracking predicts that it follows the object, and a distribution-shift artifact predicts incoherent motion. In 50 trials, 25 per direction, with the source object moved 10–15 cm, the arm drove to the recorded coordinates every time and closed on empty air at the vacated position. With two blocks swapped into each other's positions it stacks fluently in the wrong color. About a centimeter of displacement already makes the grasp fail. Without injection and with the source prompt, the policy reaches the displaced object, so the coordinate locking belongs to the injected program. After the injected sequence ends, the policy cycles fragments of its trained tasks at their trained coordinates. Outcomes are scored behaviorally, with RGB recorded for every rollout.

P Broader Impact

This work is interpretability research aimed at making VLA decision-making auditable. The artifacts we release (520+ SAE checkpoints, 79 identified concepts, the cross-task-injection rollout corpus, and the feature-exploration platform) are debugging tools and none is a deployment-ready policy. All

cross-architecture experiments run in simulation, with one physical replication on a single task pair (App. O).

Positive impact. Mechanistic decomposition supports principled failure analysis. Kill-switch mapping localizes the features and layers a VLA depends on, and spatial-binding analysis flags coordinate-grounded brittleness before deployment. The interventional protocols here (null-injection, cross-task injection, single-feature ablation) narrow a failure to a pathway, a layer, or a feature subspace.

Risks and dual use. Activation injection and feature steering redirect a deployed policy's behavior without retraining, so an attacker with white-box access to a VLA's residual stream can reroute a manipulation arm to a different target. We mitigate this exposure by releasing interpretability artifacts (SAE checkpoints, rollout videos, and the platform) and no deployable policy or robot control stack. Independently, the coordinate-bound binding documented in §4.3 is itself a deployment hazard. VLAs that pass simulation benchmarks fail under modest scene-position perturbations [Zhou et al., 2025]. Practitioners deploying VLAs for high-stakes tasks should run perturbation-style robustness benchmarks [Zhou et al., 2025, Fei et al., 2025] before deployment.

Compute and energy. Total compute and data volume are reported in App. D.7. All training used existing pretrained VLA checkpoints, and we do not pretrain new VLAs.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Each claim in the abstract and Section 1 (coordinate-bound programs, including the UR10e replication, language sensitivity, pathway specialization, and SAE pooling) is supported in Sections 4.2–4.7 with sample sizes and confidence intervals.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 4.8 and Appendix B cover benchmark scope (cross-architecture results in simulation, one UR10e replication in Appendix O), counterfactual prompt coverage, cross-task injection confounds, the coordinate-frame control we did not run, and concept identification. Appendix D.4 states which interventions replace the full layer output.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper is empirical and presents no theorems. The SAE objective (Eq. 5), the override metric (Eq. 2), and the temporal-contrastive loss (Eq. 9) are operational definitions, each validated experimentally.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 3, Appendix D, and Appendix H.2 specify SAE architecture (TopK k=64, 4×/8× expansion, tied decoder, AuxK with $\alpha_{\text{aux}} = 1/32$), training (500K samples, batch 4096, 100 epochs, lr 3×10^{-4} cosine), the injection conditions, the counterfactual prompt conditions, the two intervention-hook placements, and the temporal-contrastive matched-ablation protocol. Per-experiment sample sizes are in figure/table captions.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We provide code, SAE checkpoints, the rollout-validation harness, the feature-exploration platform (Action Atlas, https://action-atlas.com), and the public repository (https://github.com/CWRU-AISM/action-atlas). Reproduction instructions, dependency specs, and per-experiment launchers are included in the supplementary ZIP.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Hyperparameters and choices are reported in Section 3, Appendix D.1, and Appendix H.2. Episode counts per experiment type and per architecture are in Table 7. The intervention-hook protocol distinguishing full-layer-residual, residual-additive, and MLP-only placements is documented in Appendix D.4.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Small-sample and cross-model rates carry 95% Wilson intervals (Tabs. 5 and 10), comparisons use Fisher exact or two-proportion z-tests, and ANOVA effect sizes are η^2.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Compute budget is reported in Appendix D.7: 8×A100-80GB for $\pi_{0.5}$/OFT/large-scale SAE/cross-model work, RTX 5090 (32GB) for GR00T, 2×RTX 4090 (24GB) for SmolVLA/X-VLA, approximately 10,000 A100 GPU-hours in total. Total artifacts ∼7.1 TB (4.3 TB activations, 152 GB SAE checkpoints, 420,000+ rollouts).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics?

   Answer: [Yes]

   Justification: The work uses public simulation benchmarks (LIBERO, MetaWorld, SimplerEnv, ALOHA), public VLA weights, and a UR10e dataset of 335 episodes teleoperated by the authors. It involves no external human subjects, no personal data, and no scraped private content. Dual-use considerations are addressed in the Broader Impact section.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The Broader Impact section (Appendix P) discusses positive impact (failure-mode debugging tools for VLA operators, brittleness diagnostics), risks (dual-use of activation steering for unintended behavioral redirection), and recommends robustness benchmarking (e.g., LIBERO-PRO) prior to high-stakes deployment.

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse?

Answer: [Yes]

Justification: We release simulation artifacts (SAE checkpoints over LIBERO/MetaWorld/SimplerEnv activations and rollout videos) and the scored UR10e rollout videos and do not release a turn-key real-robot exploit harness. The released SAEs operate on activations of pretrained VLAs; users must obtain VLA weights separately under the original licenses.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and respected?

    Answer: [Yes]

    Justification: All five VLAs and the ACT control ($\pi_{0.5}$, OpenVLA-OFT, X-VLA, SmolVLA, GR00T N1.5, ACT), all four benchmarks (LIBERO, MetaWorld, SimplerEnv, ALOHA), and the CoTracker3 and Depth Anything 3 models used for Fig. 3 are cited with their original publications. We use checkpoints under their respective public licenses (Apache-2.0 / MIT for the simulation benchmarks; model-specific licenses per the original VLA releases).

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: We release 520+ SAE checkpoints (per-token, mean-pool, and temporal-contrastive variants across five VLAs), the rollout-validation harness for 3-arm pooling comparison, the feature-exploration platform (Action Atlas), the cross-task injection rollout corpus, and the scored UR10e rollout videos. Each is documented in the supplementary materials with directory layouts, filename conventions, and reproduction instructions.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The work involves no human subjects or crowdsourcing.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The work involves no human subjects.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?

    Answer: [N/A]

    Justification: LLM-based auto-interpretability is an optional tool in the feature-exploration platform for narrating SAE features. The core methods and claims do not use LLMs.